



車載LAN入門講座 CAN通信編

In-vehicle LAN introductory course CAN communication

第1章 CAN(Controller Area Network)とは

第2章 CANのフレーム構成

第3章 CANのエラーの仕組み

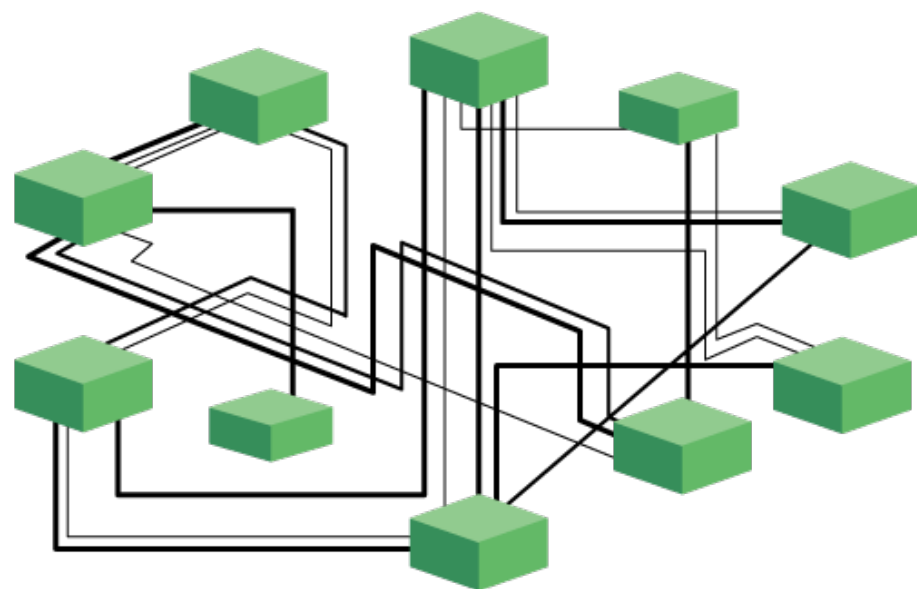
■第1章 CAN(Controller Area Network)とは

【習得内容】

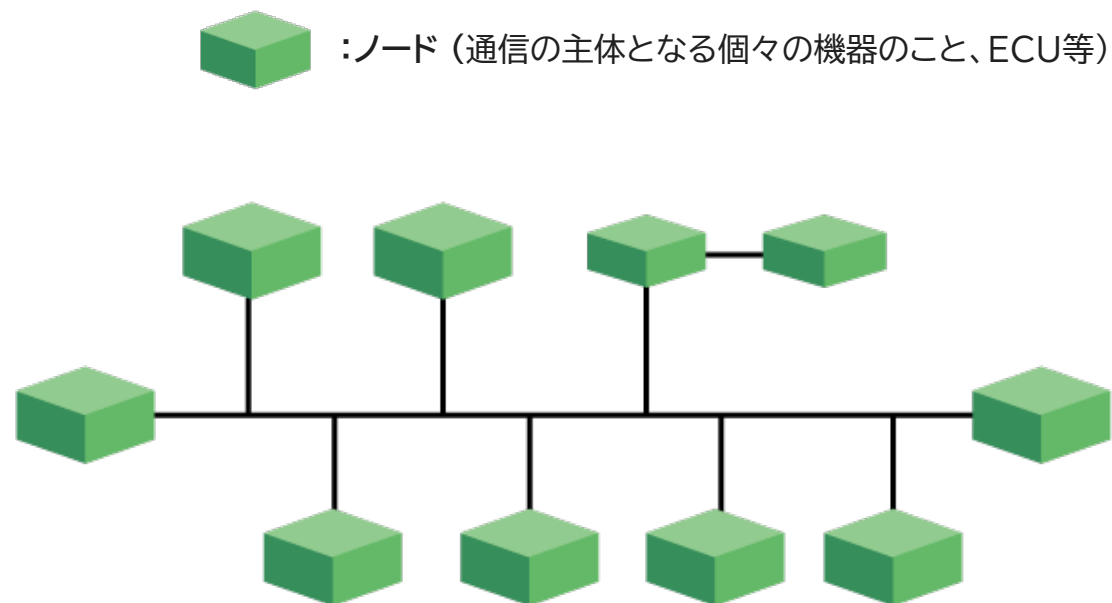
- ・CANの歴史とプロトコルの位置づけ
- ・CANの特徴
- ・CANの通信方法

- CANはController Area Networkの略
- ドイツの電装メーカーのBOSCH社が自動車の電子制御システム向けの通信プロトコルとして提唱したのが始まり
 - ワイヤーハーネスを削減する、複数のLANを介して通信する
- ISO11898で規格化され、自動車LANの標準プロトコルに位置付けられている
 - ISO11898-1:データリンク層
 - ISO11898-2:物理層 HighSpeed CAN(高速CAN)+CAN FD
 - ISO11898-3:物理層 LowSpeed CAN(低速CAN)
 - ISO11898-4:タイムトリガの規定
- 現在では、自動車業界以外にも医療、FA、産業機器など多方面で使用されている

- CANの導入によって、それまで1対1ずつで結線するしかなかった制御システムは多対多での結線が可能となった



従来のシステムの場合



CANを導入した場合

結果ワイヤーハーネスの削減を可能にした

- 通信方式 : マルチマスタ、CSMA/CA方式、半二重通信
- 伝送路 : 2線式
- 接続形態 : バス型
- 通信速度 : Max 1Mbps
- データ長 : 0Byte~8Byte
- データ誤り検出 : CRC方式

■システムの柔軟性(ノードの追加削除が容易)

- ・CANはアドレスの様な情報を持っていないため、ノードの追加/削除の際は他のノードのソフトウェア、ハードウェア、アプリケーション層に変更を加えずにシステムが設計できる

■エラーの検出 / 通知 / リカバリー機能

- ・全てのノードにエラーを検出し、他ノードへ通知する機能があり、バス全体としての通信をリカバリーする機能がある

■CANデバイス、開発ツールが豊富

- ・多くの分野でCANが使用されているため、多種多様なツールが供給されている
→コントローラ内蔵汎用マイコン、トランシーバ、開発ツール等

■自動車におけるCANの利用例

・制御系(パワートレイン系&シャーシ系制御)

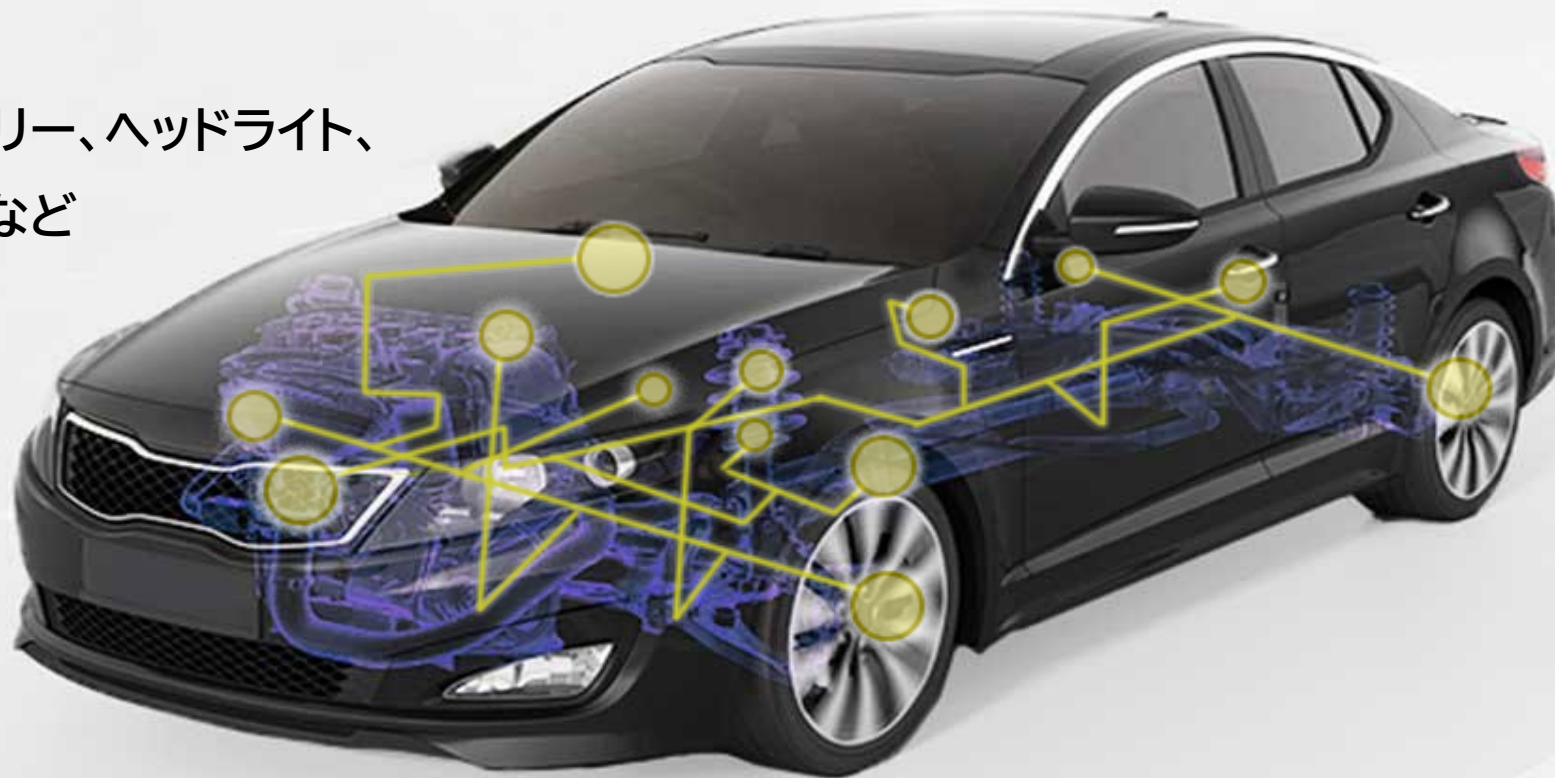
エンジン制御、トランスミッション制御、ブレーキ制御、ステアリング、EPS制御 など

・ボディ系制御

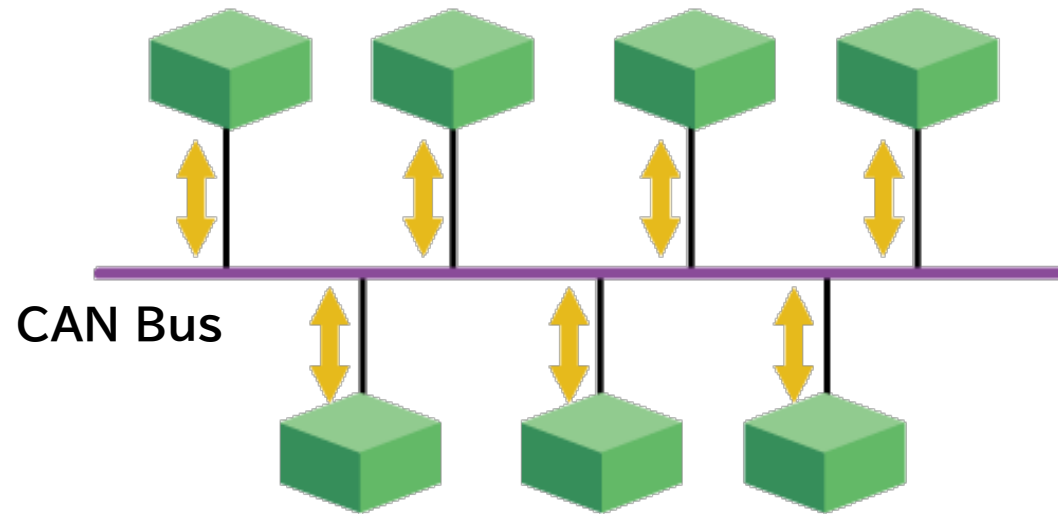
パワーシート制御、キーレスエントリー、ヘッドライト、
シート調整、ドアミラー、エアコン など

・故障診断系

ダイアグ情報の読み出し



- ・トポロジーとは、ネットワークの接続の形のことでCANはバス型が使用されている
- ・1本のメイン通信路=バスに各ノードがぶら下がる形



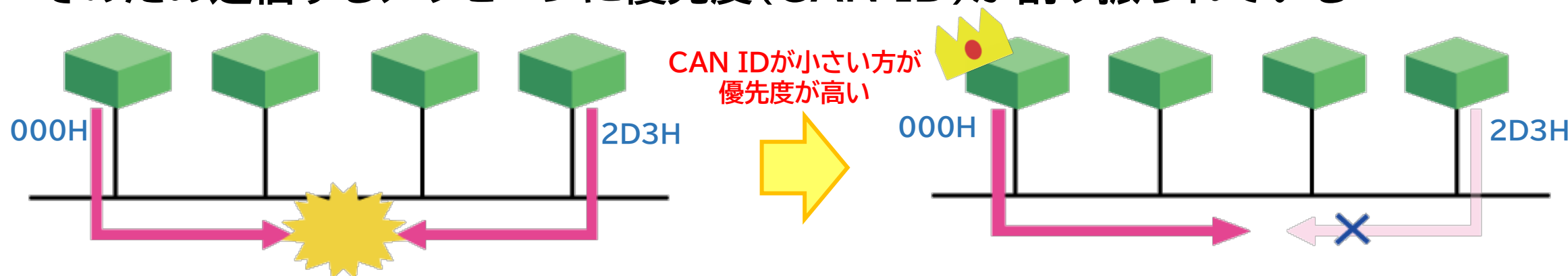
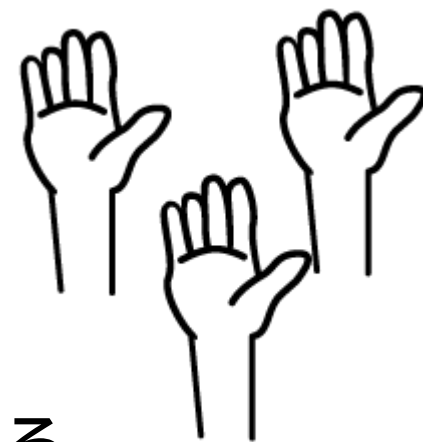
■メリット

- ・1つのノードが故障しても通信を維持できる
- ・コストパフォーマンス性が高い(ローコスト)

■デメリット

- ・1つの通信路を使用しているので、衝突回避の仕組みが必要
- ・バスがショートするとシステム全体がダウンする

- マルチマスタ方式
全てのノードは自分で送信する権利を持っている方式
CANでは送信されたメッセージは全てのノードが受信する
- CANは**CSMA/CA方式**という衝突回避方式を取っている
バスが空いているタイミングならば、どのノードでも送信を開始できる
- ただし複数のノードが同時に送信開始するとメッセージが衝突する可能性がある
そのため送信するメッセージに優先度(CAN ID)が割り振られている



メッセージが衝突しそうになった時には、優先度に従って通信調停が行われる。

■CAN対応マイコン(コントローラ内蔵)

- ・メッセージのフィルタリング、バッファリング、通信調停、エラー検知など
CANのプロトコルに従ってCPUで処理された情報の通信を行う

■CANトランシーバ IC

- ・CANコントローラから送られるデジタル信号をCANで規定されている
電圧レベルに変換、受信したメッセージの電圧レベルをデジタル信号へ変換する働き

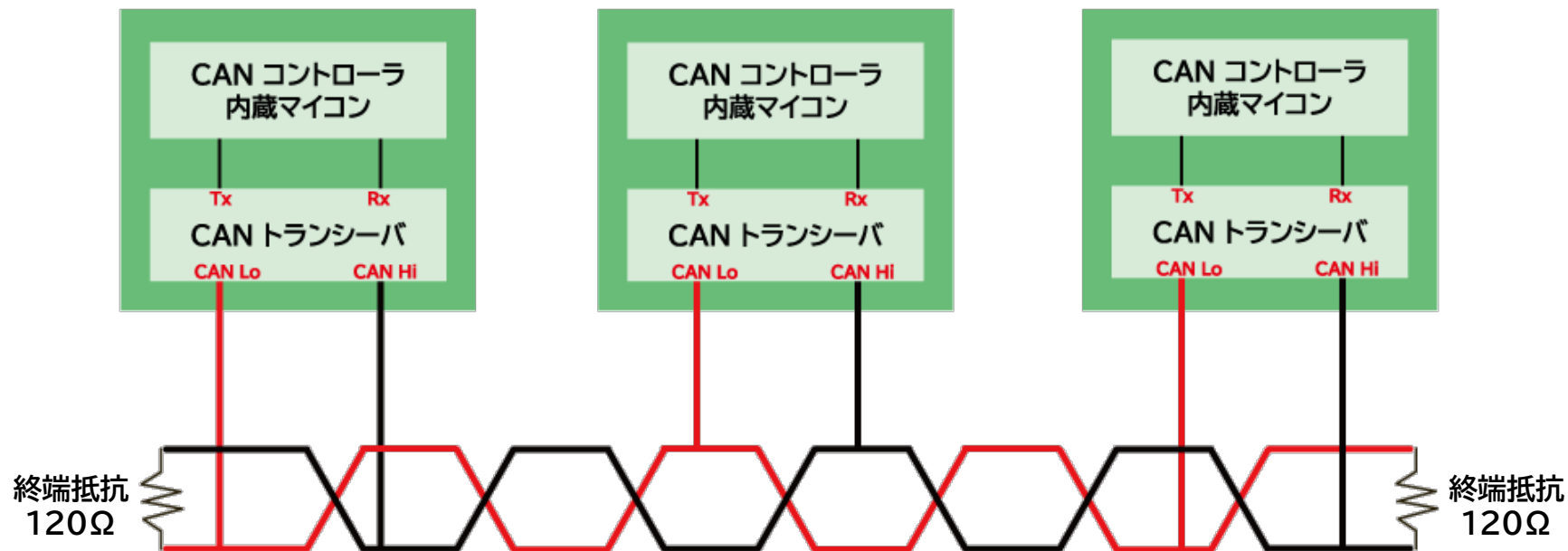
■ツイストペアケーブル

- ・2本の通信線、電線がより合わせて1対になっているケーブルで、ノイズ耐性が高い
CANでは、CAN Hi、CAN Loという2本の線がより合わさっている

■終端抵抗

- ・信号の反射を防止する

- CANはCAN Hi、CAN Loとよばれる2本の通信線を使って、送信受信の両方を行っている。
- ISO11898-2では配線の両端に120Ωの終端抵抗を2つ設置する必要がある。

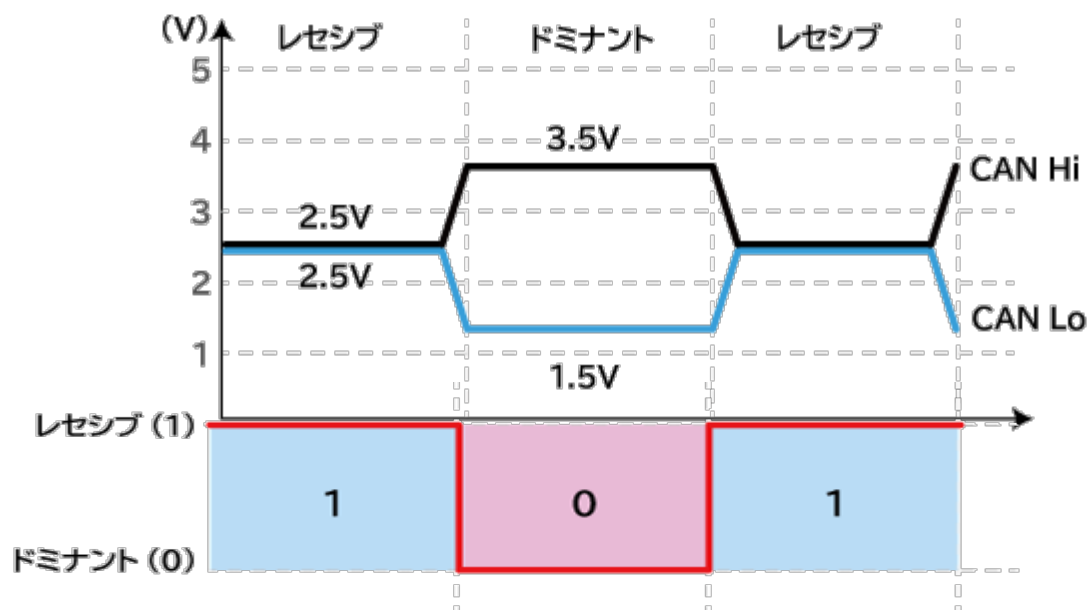


ISO11898-2(Highspeed CAN)の接続イメージ

・CANの信号は2本の通信線の電圧差によって送信される(2線式差動電圧方式)

■電位差がある時 :ドミナントと呼び論理値0を指す

■電位差がない時 :レセシブと呼び論理値1を指す



ISO11898-2(HighspeedCAN)の場合

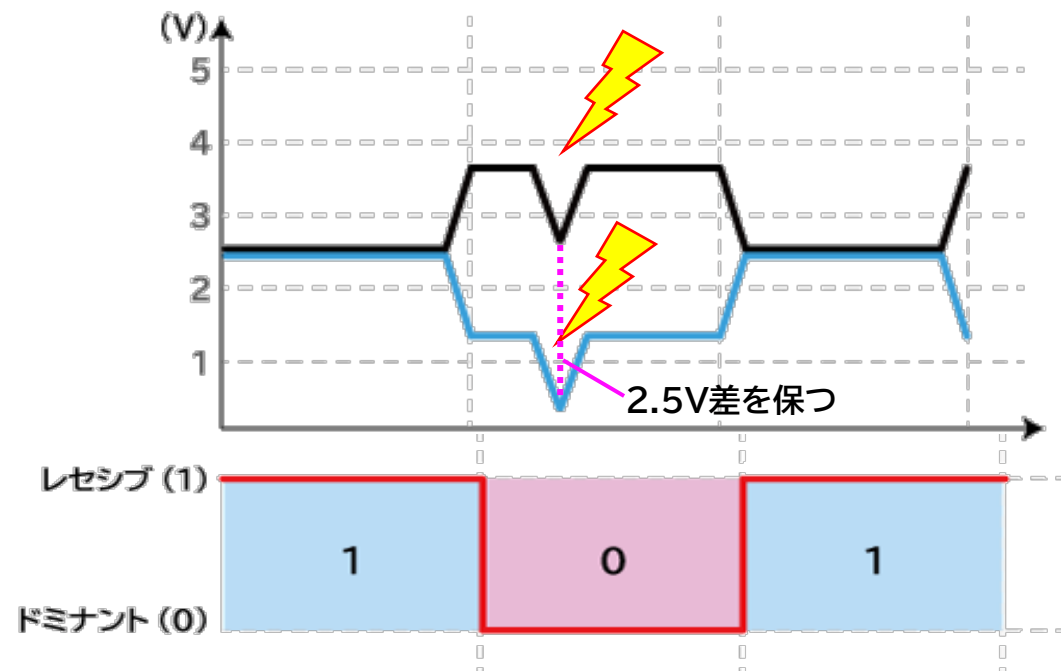
バスの状態	レセシブ			ドミナント		
	Min	Ave	Max	Min	Ave	Max
CAN Hi	2.00	2.50	3.00	2.75	3.50	4.50
CAN Lo	2.00	2.50	3.00	0.50	1.50	2.25
電位差 (Hi-Lo)	-0.50	0	0.05	1.50	2.00	3.00

ドミナントはレセシブより優先される

- CAN Hi、CAN Loの2線が何故平行線ではなくツイストしている必要があるのか

■ノイズへの耐性

- ノイズが発生した場合でもCAN Hi、CAN Loの両方に同じタイミングでノイズが乗るため、電位差は一定を保たれる
- 信号レベルでは、正しく判定される
- ツイストしていない2本線では、同時にノイズが乗らないため電位差に異常が発生し、信号レベルも乱れてしまう(データ化け)
- エミッションやイミュニティに優れており、ノイズが発生しにくく、誤動作が起きにくい様になっている
→ツイストケーブルでは、隣合う節で発生する電流や磁束が逆方向になるため、それぞれを打ち消し合うことができる。



■第1章 まとめ

- CANは**ISO11898**によって規定されている
- 多重通信で共通のバスを使った**1対他の通信を実現**(省線化)
- **マルチマスタ方式**で衝突回避は**CSMA/CA方式**を採用
- 通信速度は**最大1Mbps**まで、データは**最大8byte**まで送信可能
- 通信はCAN Hi、CAN Loの2線を使った**2線式差動電圧方式**を採用し、**電位差がある時をドミナント、電位差がない時をレセシブ**と呼ぶ
- 配線の両端には**120Ωの終端抵抗を2つ設置**する必要がある
- CAN通信の使い勝手の良さと信頼性の高さから**自動車分野以外にも活用**されている

■第2章 CANのフレームの構成

【習得内容】

- ・CANで使用されるフレームの種類と構成
- ・フレームの使用方法
- ・フレームの処理に関するルール

0:フレームの種類

1:データフレーム

2:リモートフレーム

3:エラーフレーム/オーバーロードフレーム/
インターフレームスペース

4:ビットタイミング

- CANのフレームフォーマットには以下の4種類とインターフレームスペースがある

■データフレーム :送信ノードから受信ノードへメッセージを送る時に使用

■リモートフレーム :特定の送信ノードに対してメッセージを要求する時に使用

■エラーフレーム :エラーを検出したノードが他へエラー通知する時に使用

■オーバーロードフレーム :受信待機状態になっていない事を通知する時に使用

【インターフレームスペース】

- データフレームおよびリモートフレームを前のフレームと分離させる時に使用

- ・データフレームとリモートフレームは標準フォーマットと拡張フォーマットがある

■標準フォーマット : IDを11bit分まで設定できるフレームフォーマット

■拡張フォーマット : 標準フォーマットよりも設定できるIDの数が多い

0:フレームの種類

1:データフレーム

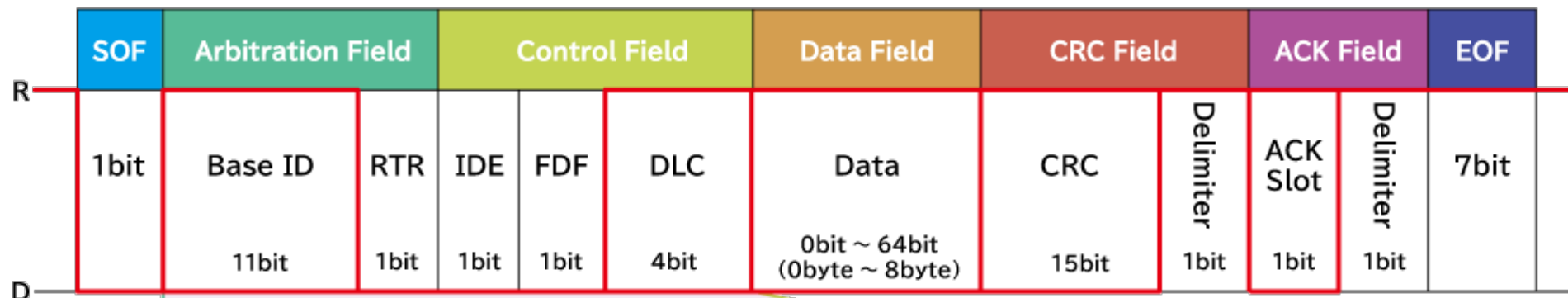
2:リモートフレーム

3:エラーフレーム/オーバーロードフレーム/
インターフレームスペース

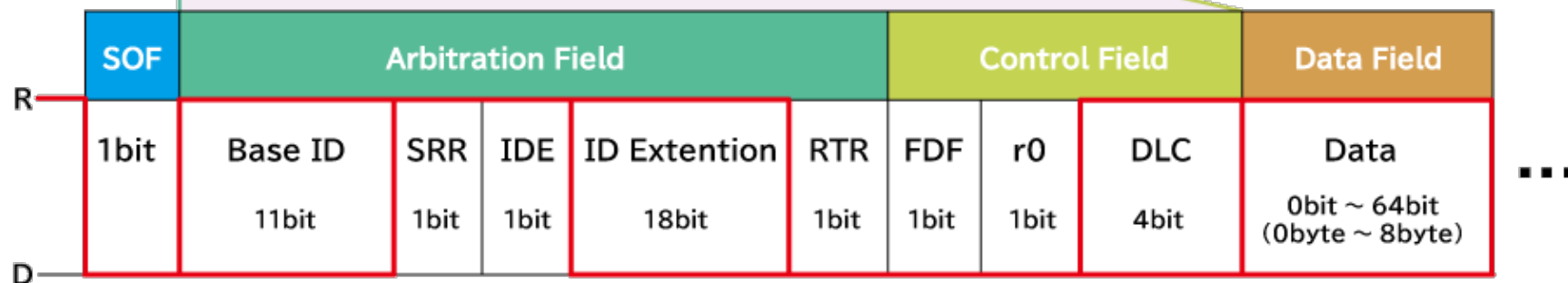
4:ビットタイミング

- ・ユーザが設定してデータの転送を行う時に使用するフレーム
- ・7つのフィールドで構成されている

■標準フォーマット

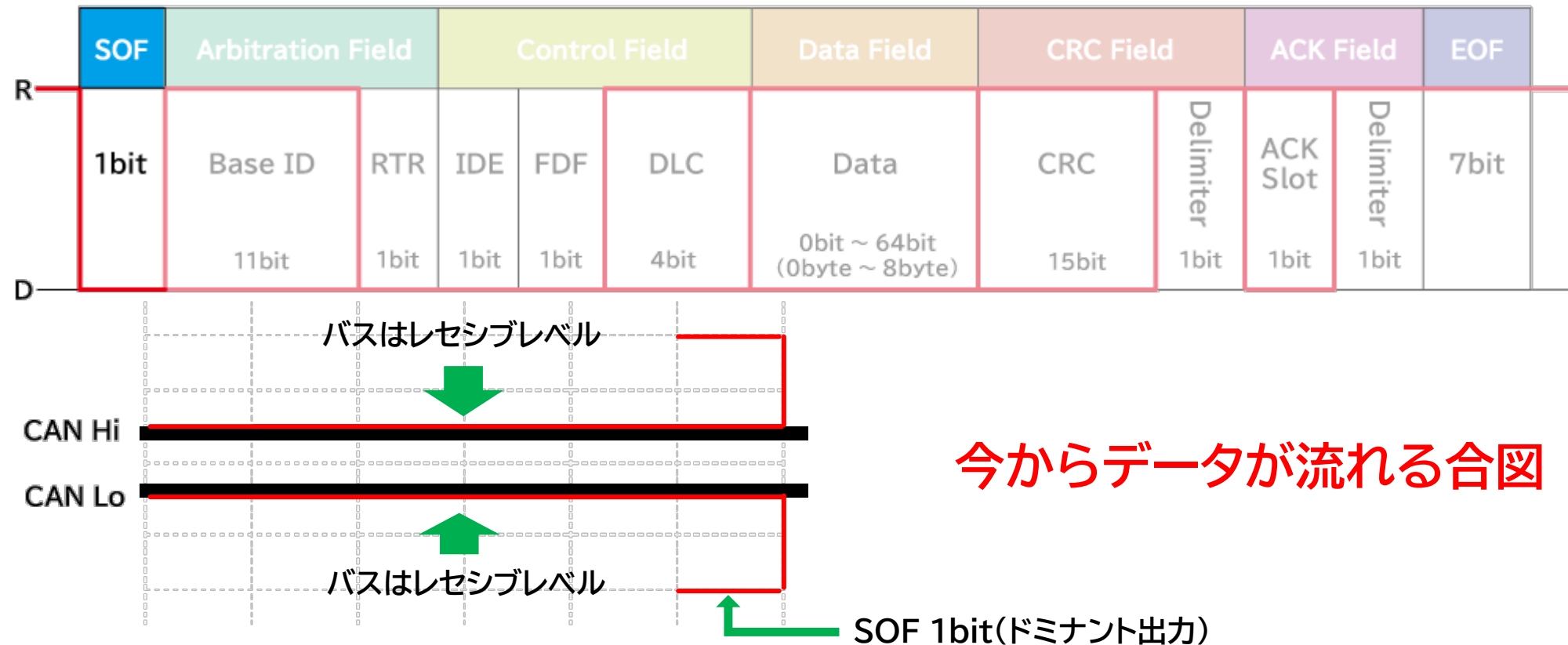


■拡張フォーマット



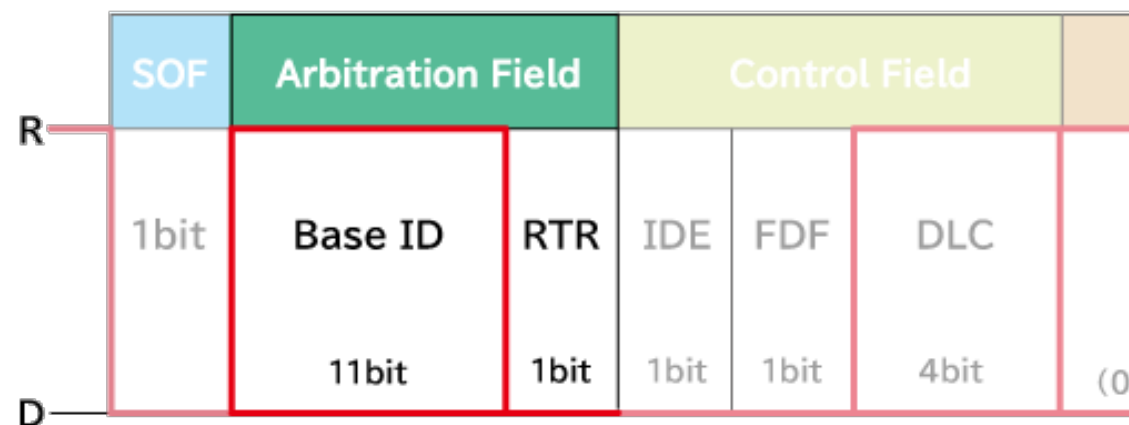
拡張フォーマットで影響のあるフィールドは、
アービトレーションフィールドとコントロールフィールドの2つ

- 何もメッセージが流れていない時のバスは常にレセシブ状態。
- データフレームの送信を開始する時には、1bitのSOF(Start Of Frame)をドミナントで送信する。



- フレームの優先順位を判断する領域
- CANのメッセージにはIDが割り振られている。
- 標準フォーマットではBase IDで**11bit分のID**が設定可能。

■標準フォーマット



IDの値でメッセージの優先度を決定している

- RTR :データフレームとリモートフレームの判別を行う。**データフレームは常にドミナント**

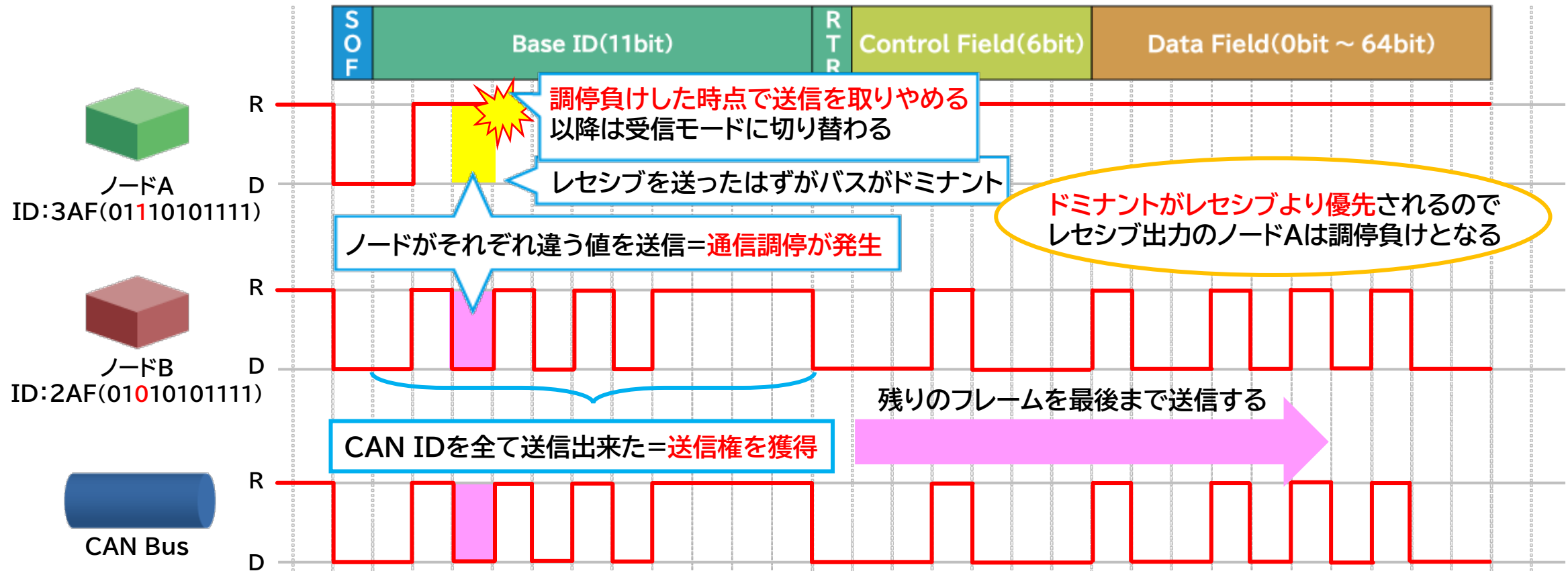
【通信調停：アービトレーションとは】

- ・同時に2つ以上のノードが送信を開始した時に、1bit単位で値を比較し、メッセージの優先順位を決定すること

【通信調停の仕組み】

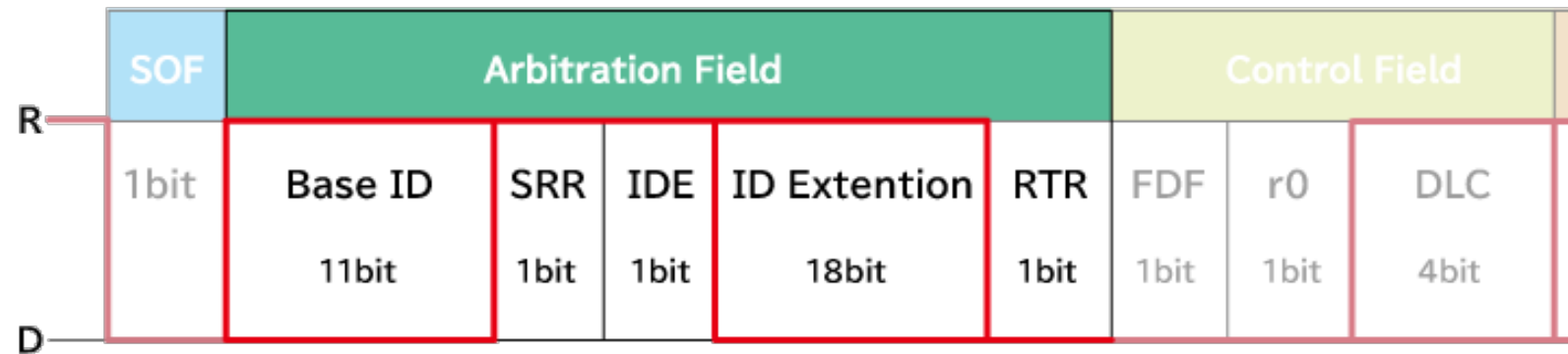
- ・CANでは、バスアイドル中は早く送信したノードが送信権を獲得する。
- ・複数のノードが同時に送信を開始した時は、各メッセージのアービトレーションフィールドにあるIDを用いて優先度を決定する。
- ・各送信ノードは自分が送信したIDとバス上のレベルを1bit単位で比較し、ドミナントを長く出し続けたノードが送信権を獲得する。
- ・通信調停に負けたノードは次のbitから送信動作を停止し受信動作に切り替わる。

【通信調停の仕組み】



- ・拡張フォーマットは標準ID11bitに加えて、更に18bit分を設定可能

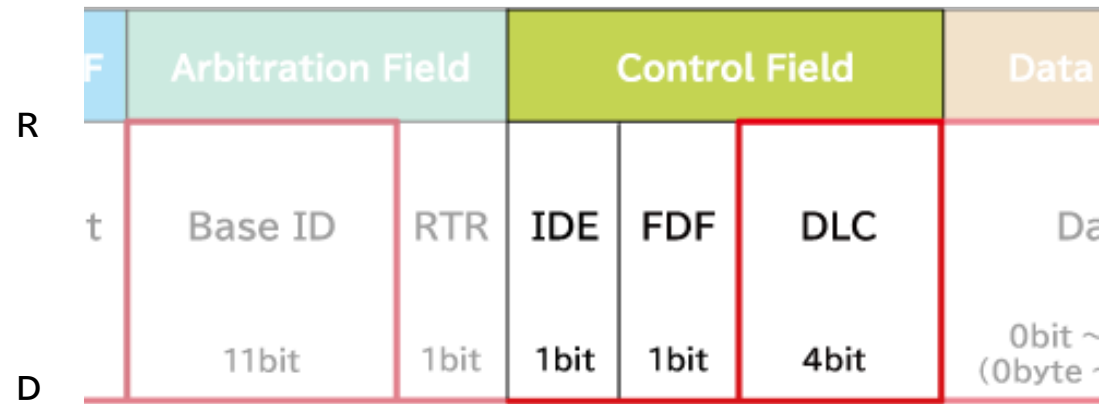
■拡張フォーマット



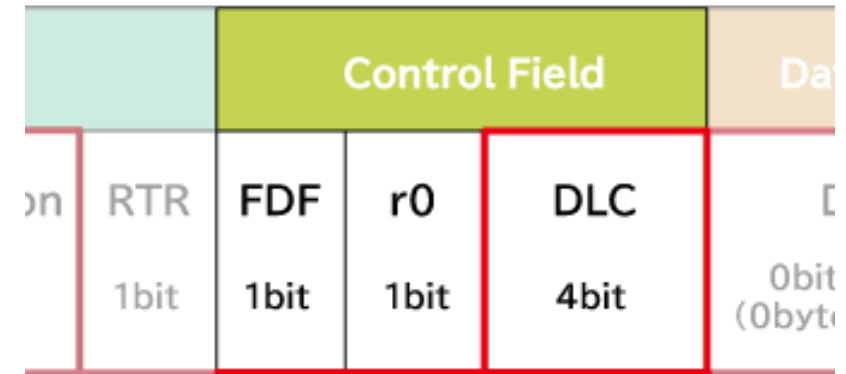
- ・Base ID :標準11bit分のIDを設定
- ・SRR :Substitute Remote Request Bitで**1bitのレセシブ固定**
- ・IDE :Identifier Extension Bitで**1bitのレセシブ固定**
標準フォーマット、拡張フォーマットの区別を行う
- ・ID Extension :18bit分の拡張IDを設定、Base IDと併せて**計29bit分のIDを設定可能**
- ・RTR :データフレームとリモートフレームの判別を行う。**データフレームはドミナント**

- ・フレームに含まれるデータのサイズとフォーマットの種別を行う

■標準フォーマット



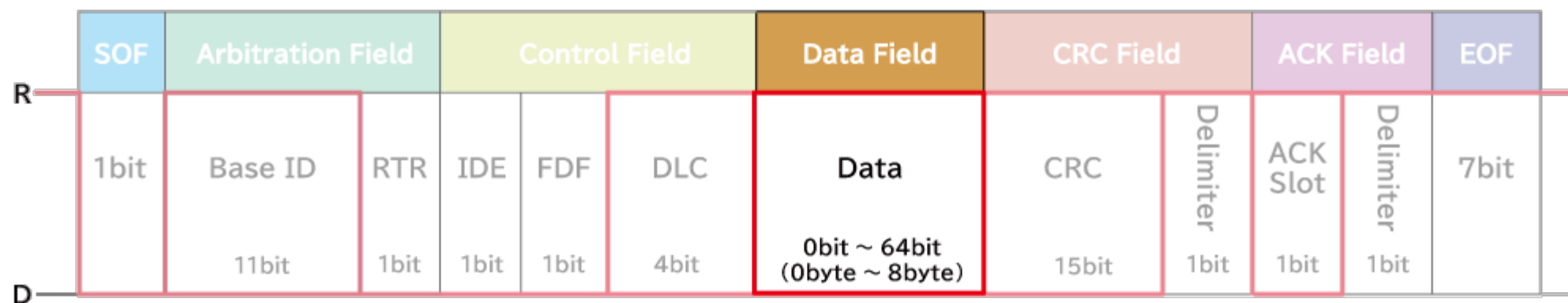
■拡張フォーマット



- ・IDE : 標準フォーマット、拡張フォーマットの区別を行う。標準フォーマットは1bitのドミナント固定
- ・FDF : CANと、CAN-FDを判別するbit。CANではドミナント固定
- ・DLC : データ長の設定を行う
- ・r0 : 予約領域で1bitのドミナント固定(拡張フォーマットのみ)

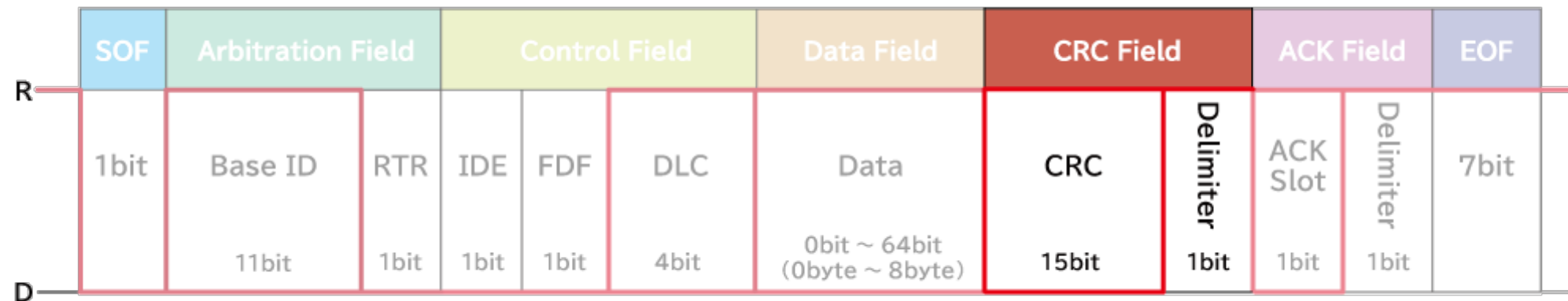
- データの実態を格納する
- データ長は0byte～8byteで、コントロールフィールドでデータ長を指定する。

■標準フォーマット



- ・フレームの伝送誤り検出用のCRCを表すフィールド
- ・送信ノード、受信ノードそれぞれで、SOF～データフィールドまでの値を演算し、その結果を比較することで、伝送誤りを判断する

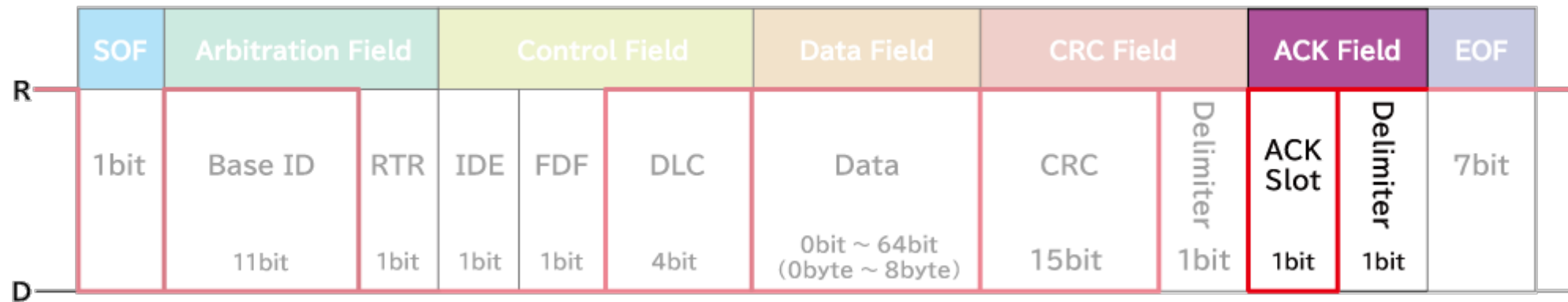
標準フォーマット



- ・CRC :SOF～データフィールドまでの演算した値を格納する
- ・CRC Delimiter :1bitのレセシブ固定

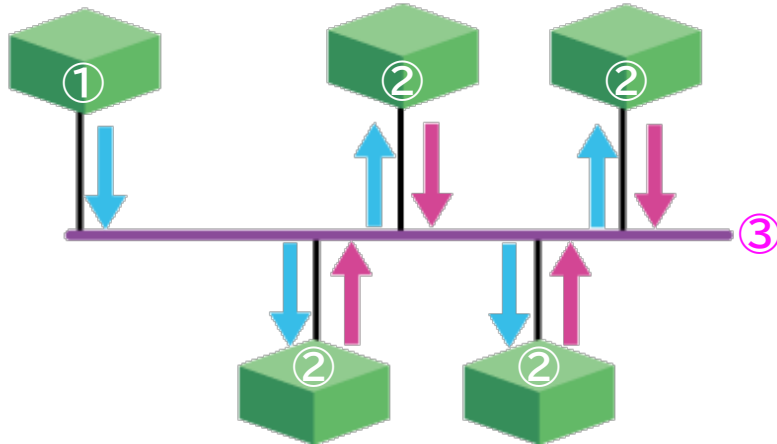
- ・正常にメッセージを受信できたかどうかを確認するフィールド
- ・送信側はACKフィールドの部分を1bitのレセシブで送信し、受信側はこの1bitをドミナントで送信する

■標準フォーマット



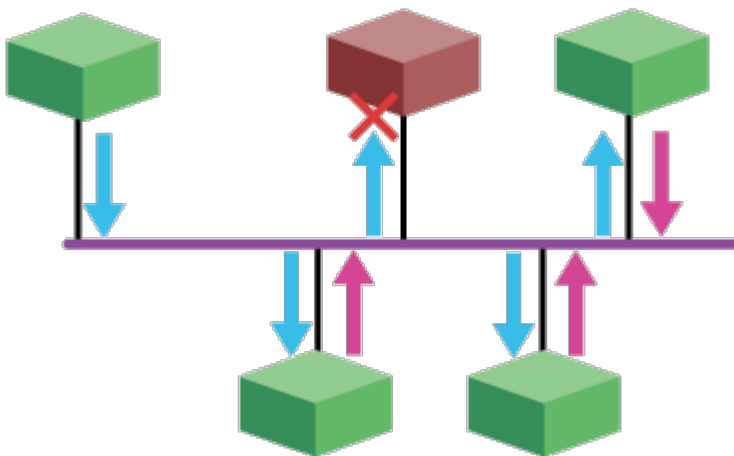
- ・ACK Slot :送信ノードはレセシブで送信する。
メッセージを正常に受信したノードはドミナントを送信する。
- ・ACK Delimiter :1bitのレセシブ固定

■ACKの仕組み



- ①送信ノードはACK Slotをレセプブで送信する
- ②正常にメッセージを受信したノードはACK Slotをドミナントにして返信する
- ③ドミナントが送信されたことでバスのレベルはドミナントになる

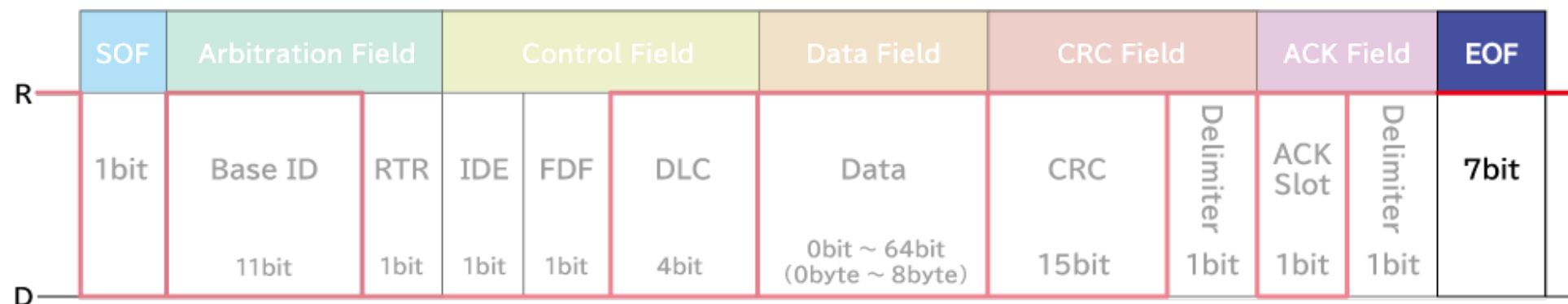
■注意点



- ・正常に受信できなかったノードはACKを返さない
- ・1つのノードでも正常に受信できていれば、そのノードがドミナントを送信するため、バスのレベルはドミナントになる。
- ・バス全体としては正常にメッセージを受信したと判断される
=どのノードが正常受信できたかまでは判別できない
- ・どのノードもACKを返さなければ、ACKエラーとなる

- ・フレームの終了をあらわす7bitのレセシブ

■標準フォーマット



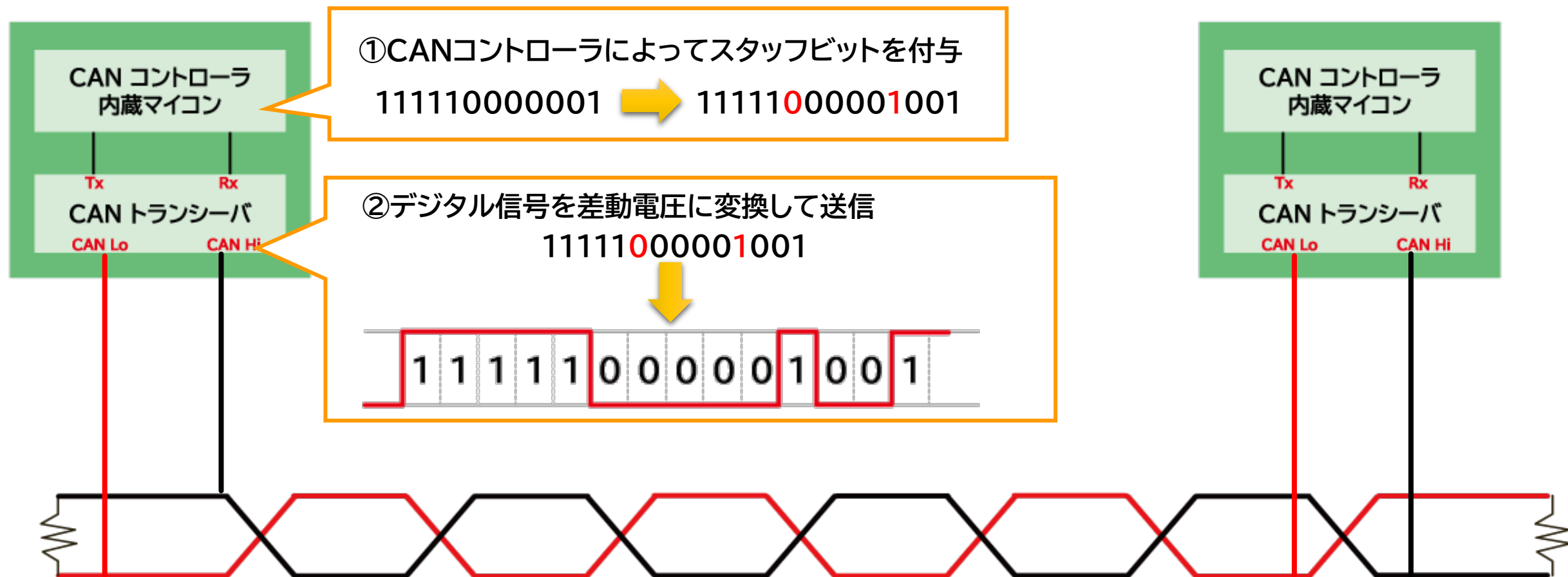
- CANは信号の立ち下がりを検出することで、他ノードと同期を取っている。
- 1または0の信号が長期間継続すると送信ノードと受信ノード間にタイミング誤差が累積し、同期ずれに繋がる。
- 同期ずれを防ぐためにCAN通信では5連続した0または5連続した1を送信したら、その反転ビットを挿入しなければならない仕様になっている。
- この仕様をビットスタUFFィングといい、挿入されるビットをスタUFFビットという。

■ビットスタUFFィング2/2-1

-bit stuffing-

■送信するデータ:1111100000001

■受信したデータ:1111100000001



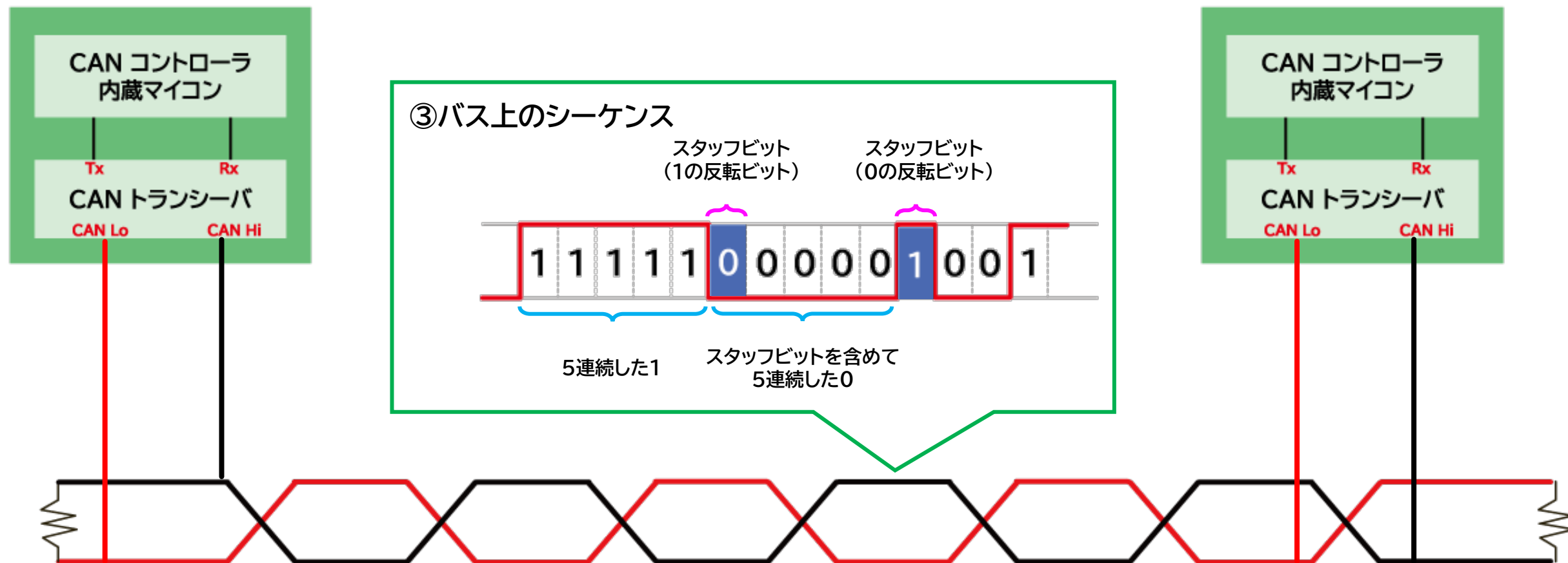
- スタUFFビットはシーケンスレベルでCANコントローラによって自動で付与、除去が行われる。
- 直接CANの波形をオシロスコープ等で確認する場合にはスタUFFビットが入っているので注意が必要。

■ビットスタUFFィング2/2-2

-bit stuffing-

■送信するデータ:111110000001

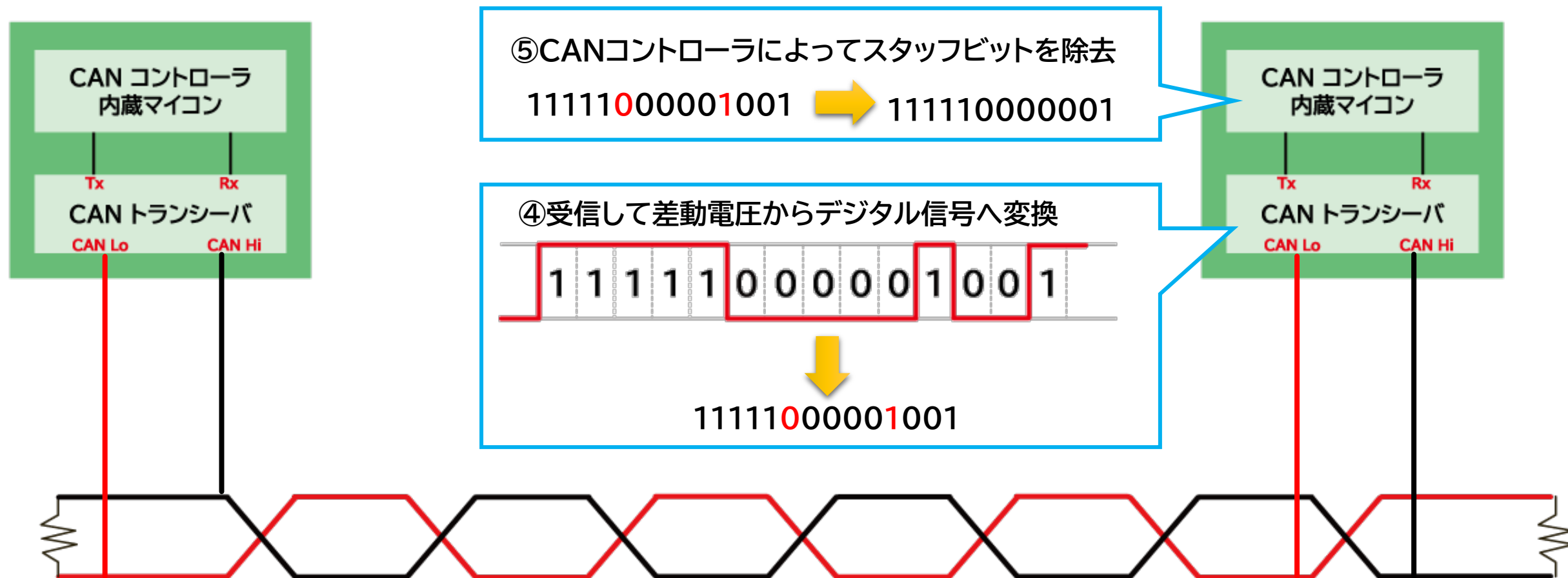
■受信したデータ:111110000001



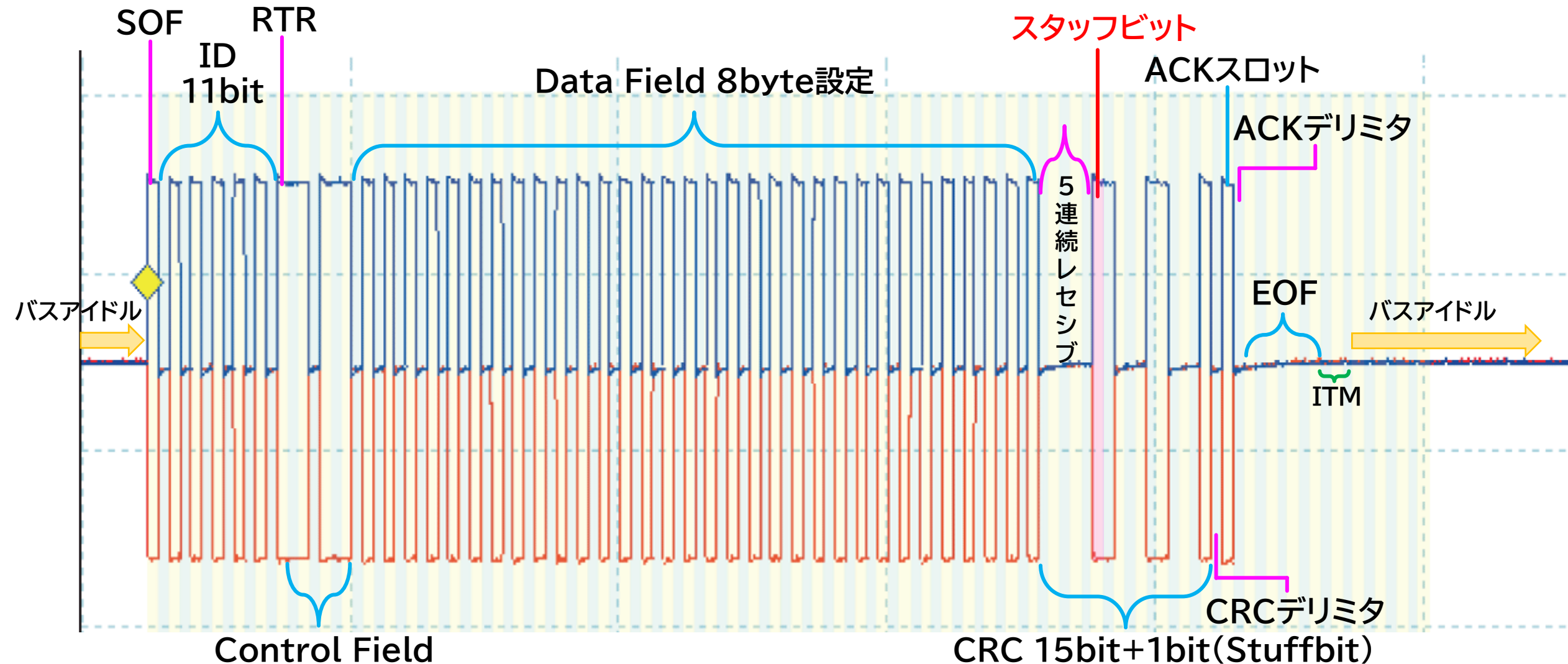
- スタップビットはシーケンスレベルでCANコントローラによって自動で付与、除去が行われる。
- 直接CANの波形をオシロスコープ等で確認する場合にはスタップビットが入っているので注意が必要。

■送信するデータ:1111100000001

■受信したデータ:1111100000001



- スタUFFビットはシーケンスレベルでCANコントローラによって自動で付与、除去が行われる。
- 直接CANの波形をオシロスコープ等で確認する場合にはスタUFFビットが入っているので注意が必要。



0:フレームの種類

1:データフレーム

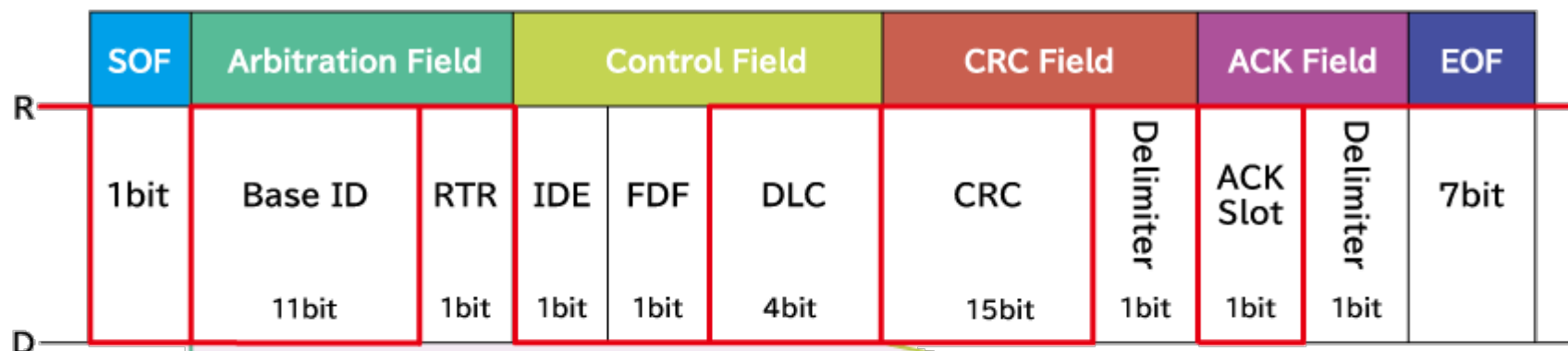
2:リモートフレーム

3:エラーフレーム/オーバーロードフレーム/
インターフレームスペース

4:ビットタイミング

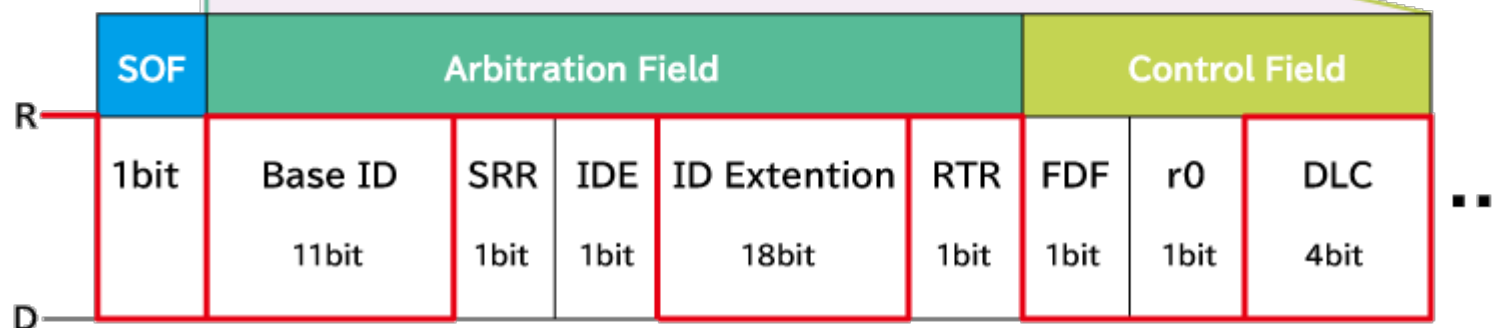
- ・リモートフレームは受信ノードから送信ノードへメッセージを要求する時に使用する。
- ・リモートフレームのフィールド構成は、データフレームからDataFieldを除いたもの。

■標準フォーマット



拡張フォーマットで影響のあるフィールド

■拡張フォーマット



拡張フォーマットで影響のあるフィールドは、
アービトレーションフィールドとコントロールフィールドの2つ

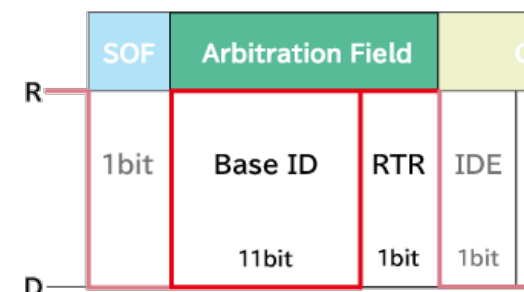
■SOF

- ・データフレームと同じで、メッセージの開始を表す1bitのドミナント

■Arbitration Field

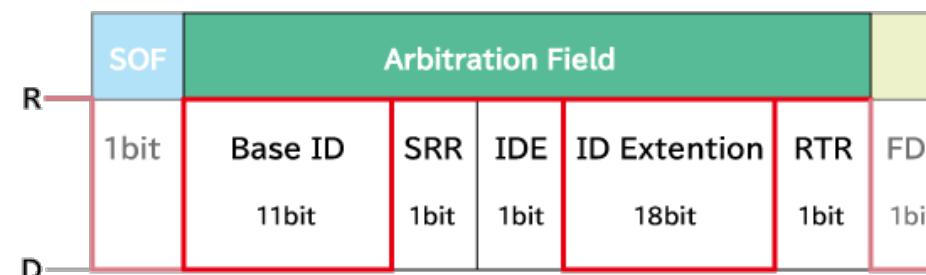
【標準フォーマット】

- ・Base ID : 要求するメッセージのIDを表す。
- ・RTR : データフレームとリモートフレームを判別を行う。リモートフレームはレセシブ固定



【拡張フォーマット】

- ・BaseID : 要求するメッセージのIDを表す。
- ・SRR : 1bitのレセシブ固定
- ・IDE : 1bitのレセシブ固定
- ・ID Extention : 要求するメッセージのIDを表す。
- ・RTR : データフレームとリモートフレームを判別を行う。リモートフレームはレセシブ固定



■Control Field

【標準フォーマット】

- ・IDE : 標準フォーマットと拡張フォーマットの判別に使用、標準フォーマットはドミナント
- ・FDF : CANと、CAN-FDを判別するbit。CANではドミナント固定
- ・DLC : **要求するメッセージのデータ長**を表す。

【拡張フォーマット】

- ・FDF : CANと、CAN-FDを判別するbit。CANではドミナント固定
- ・r0 : 予約領域、ドミナント固定
- ・DLC : **要求するメッセージのデータ長**を表す。

■CRC Field

- ・データフレームと同等、伝送誤り検出に使用する

■ACK Field

- ・データフレームと同等、正常にメッセージを受信できたかどうかを確認する

■EOF

- ・データフレームと同等、メッセージの終わりを表す7bitのレセシブ

0:フレームの種類

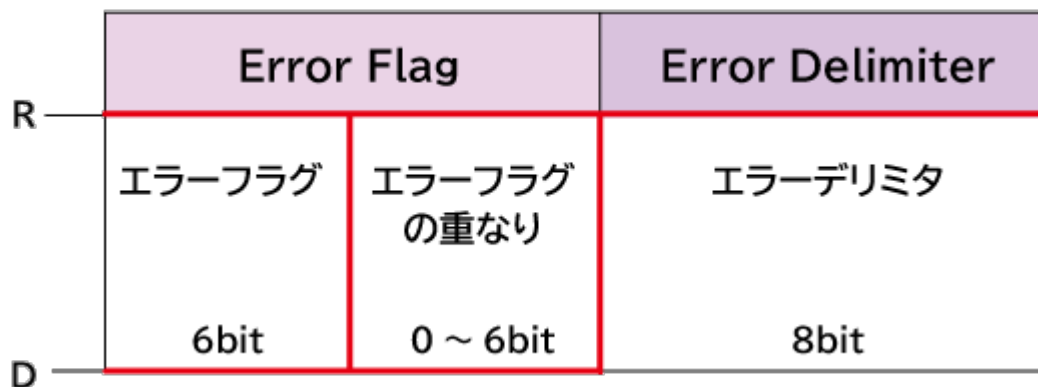
1:データフレーム

2:リモートフレーム

3:エラーフレーム/オーバーロードフレーム/
インターフレームスペース

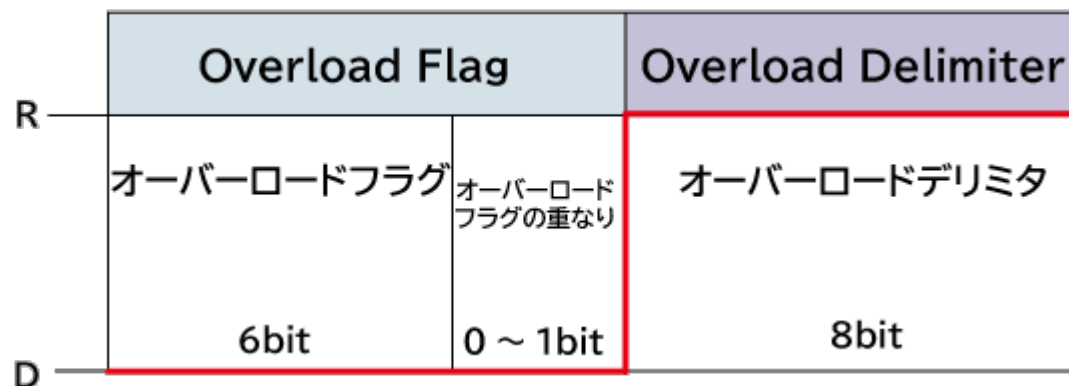
4:ビットタイミング

- エラーを検出したノードがエラーを通知するために使用するフレーム
- エラーフレームは単独で存在せず、**他のフレームに寄生する形で送信される。**



- エラーフラグ : エラーを送信するノードの状態によって内容が変化する
 - アクティブエラーフラグ: 6bitのドミナント
 - パッシブエラーフラグ: 6bitのレセシブ
- エラーフラグの重なり : 複数のノードが送信するエラーフラグが重なる部分
- エラーデリミタ : エラーフレームの終了を表す8bitのレセシブ

- ・受信待機状態になっていないことを通知する
- ・EOF後2bit以内にドミナントが検出された時に通知される



- ・オーバーロードフラグ : 6bitのドミナントで表す。
- ・オーバーロードフラグの重なり : 複数のノードが送信するオーバーロードフラグが重なる部分
- ・オーバーロードデリミタ : オーバーロードフレームの終了を表す8bitのレセシブ

- ・フレーム間で確保しなければならないスペース
- ・データフレームとリモートフレームの前に置くことでフレームを分離する。
エラーフレームとオーバーロードフレームの前には挿入されない。

■通常のインターフレームスペース



■エラーパッシブ状態の時のインターフレームスペース



- ・インターミッション : 3bitのレセシブで、この間にドミナントが検出された時にオーバーロードフレームを通知する
- ・サスペンドトランスミッション : 直前のメッセージを送信したノードがエラーパッシブ状態の時に追加される8bitのレセシブ区間
- ・バスアイドル : 次のフレームまでの間、バスが空いている状態。

0:フレームの種類

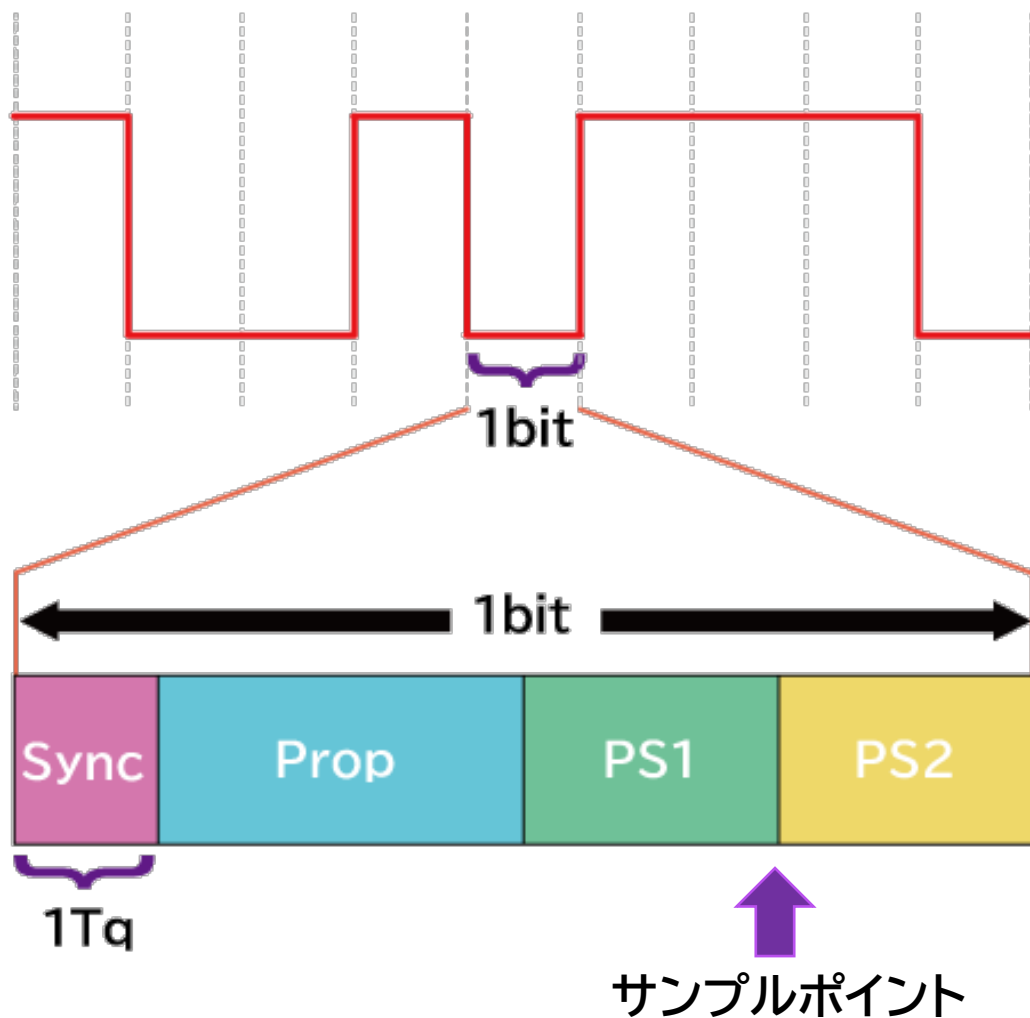
1:データフレーム

2:リモートフレーム

3:エラーフレーム/オーバーロードフレーム/
インターフレームスペース

4:ビットタイミング

- CAN 通信では1ビットが4つのセグメントから構成されている



■Tq(1 Time quantum)

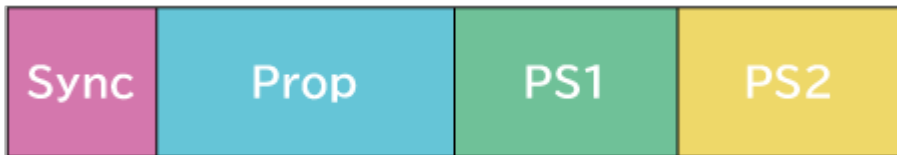
- 各セグメントを構成する最小単位

$$1Tqの時間 = 1 / (1bit内のTq数 \times \text{ビットレート(bps)})$$

■サンプルポイント

- ビットの値を判断するポイント。
この位置でのバスレベルが そのビットの値となる

※ 同一ネットワークにあるノードは全て同じサンプルポイントに設定すること



■Sync_Seg(シンクロナイズーションセグメント)

- ・バスにつながるノードが送受信のタイミングを合わせるための区間
各ビットの始まりを表す。1Tqで構成される

■Prop_Seg(プロパゲーションタイムセグメント)

- ・ネットワーク上の物理的な遅延を吸収するための区間

■Phase_Seg1(フェーズバッファセグメント1)

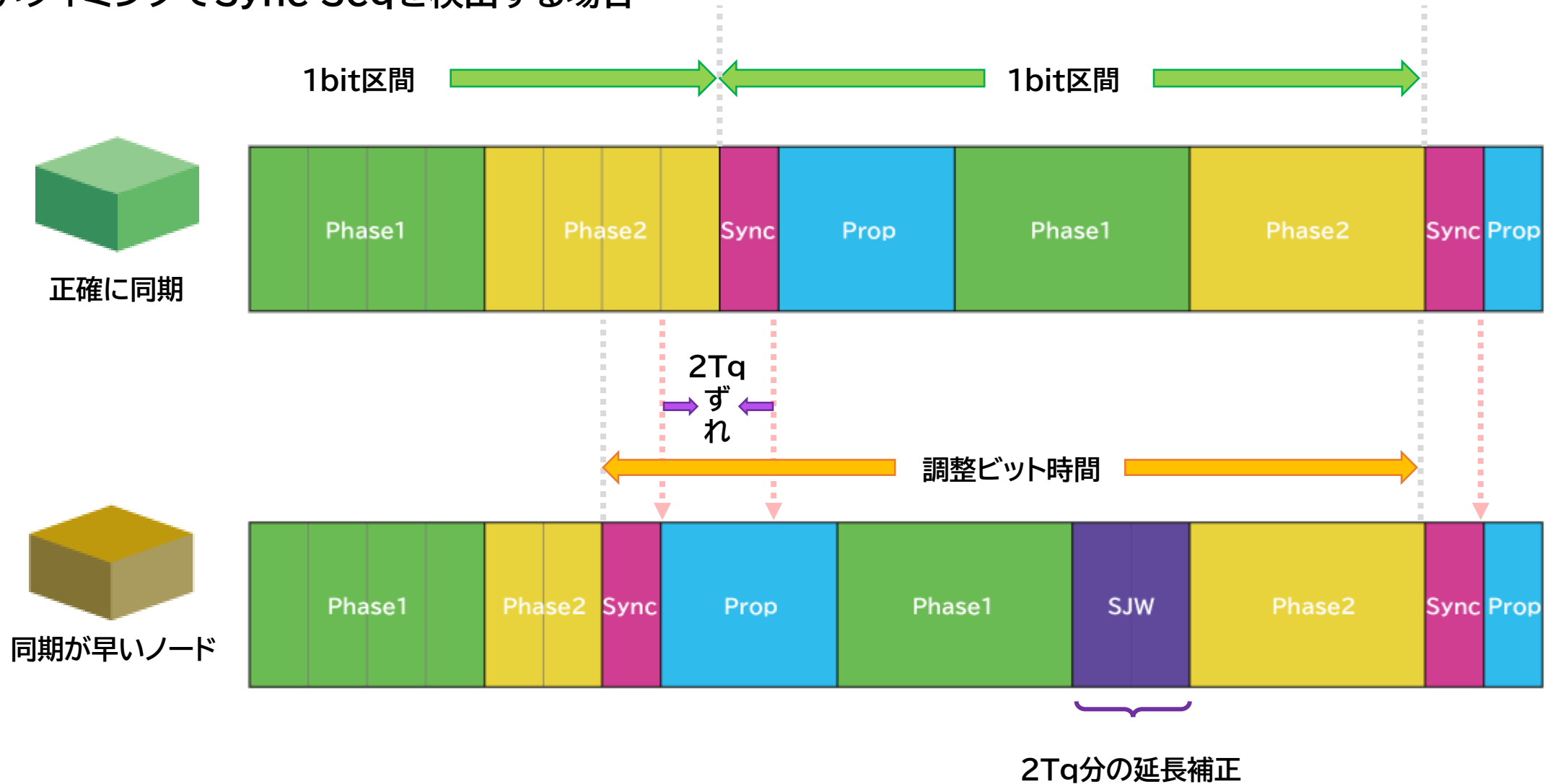
■Phase_Seg2(フェーズバッファセグメント2)

- ・Sync_Segで同期合わせが出来なかった場合に、そのずれを調整する区間
- ・ずれの調整はSJWの幅を加減して行う
- ・Phase_Seg1の直後がサンプルポイントとなる

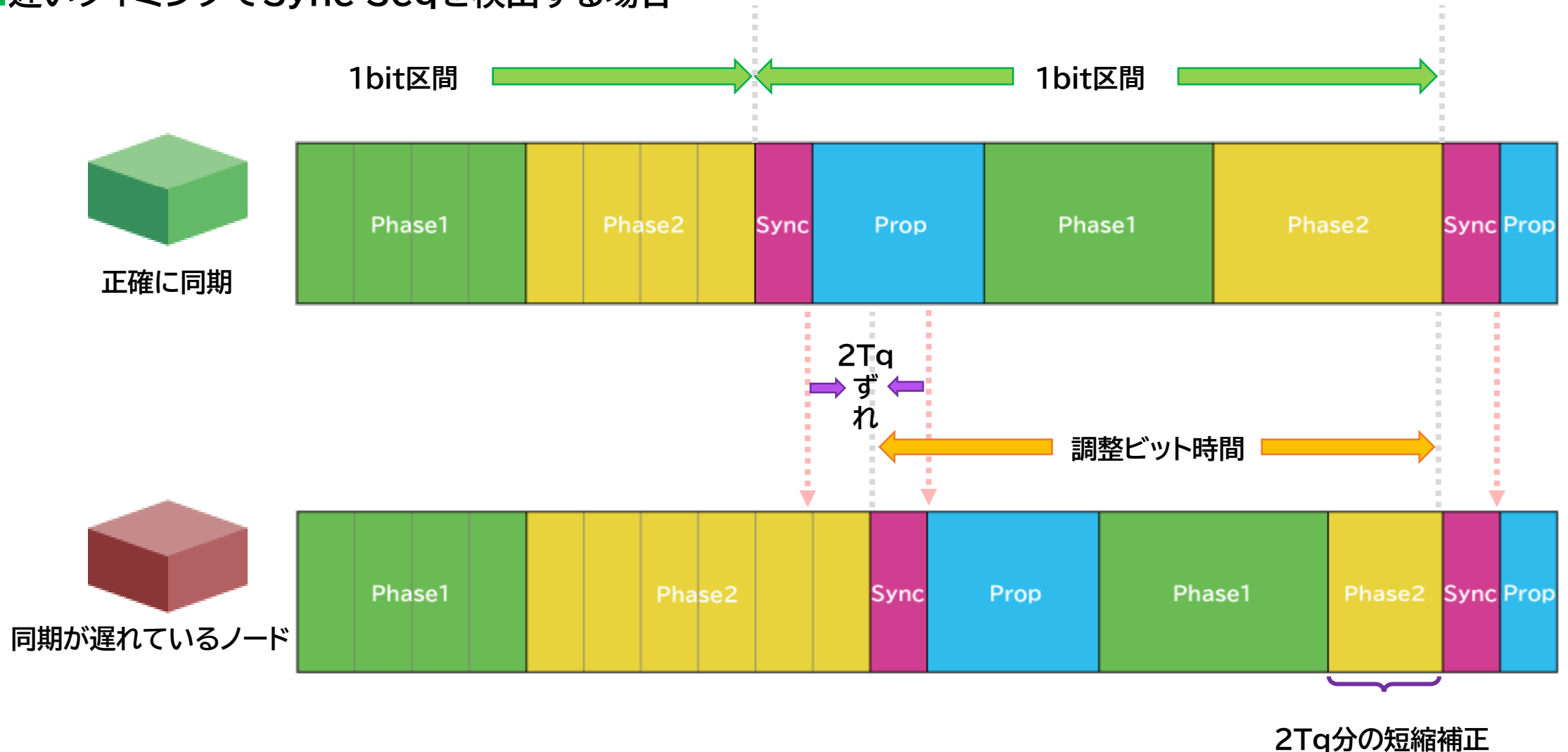
■SJW(リシンクロナイズーションジャンプ幅)

- ・再同期によるずれを調整する幅のこと、CANコントローラで設定する

■早いタイミングでSync Seqを検出する場合



■遅いタイミングでSync Seqを検出する場合



■第2章 まとめ

- CANは4種類のフレームとインターフレームスペースを使って通信を行う
- データフレームとリモートフレームには、標準フォーマットと拡張フォーマットがあり、拡張フォーマットではCAN IDの設定できる数が増える。
- フレームが同時に送信された時には、アービトレーションフィールドにて、通信調停を行いCAN IDを比較して判定している
- CANでは5ビット連続したレベルが続くとスタUFFビットを挿入する必要がある
- フレームとフレームの間では、ある一定の間隔が確保される必要がある
- CANは1bit内が4つのセグメントから構成されており、同期動作・遅延補正・サンプルポイントの設定等のバス管理機能が実装されている。

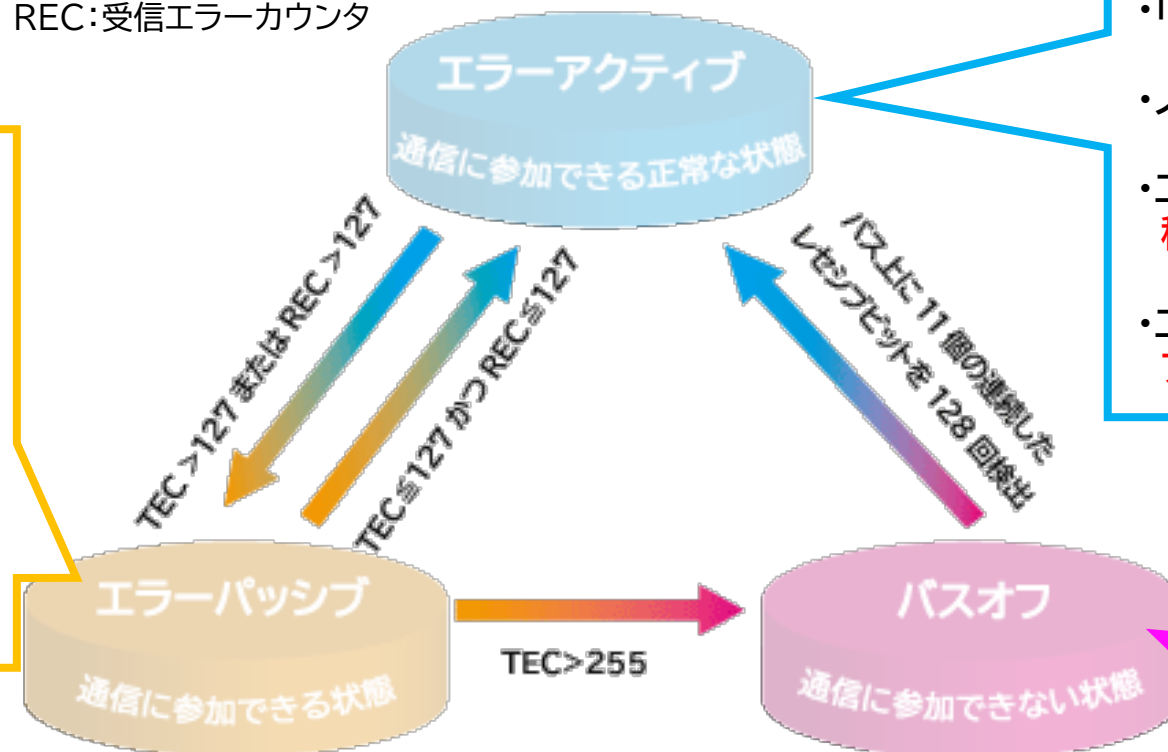
■第3章 エラーの仕組み

【習得内容】

- ・エラーの状態遷移
- ・エラーの種類と検出条件
- ・エラーフレームの送信シーケンス

- ・各ノードはエラー状態というステータスをもっている
- ・各ノードの持つ送信エラーカウンタ、受信エラーカウンタの値に応じていずれかに遷移

TEC:送信エラーカウンタ
REC:受信エラーカウンタ



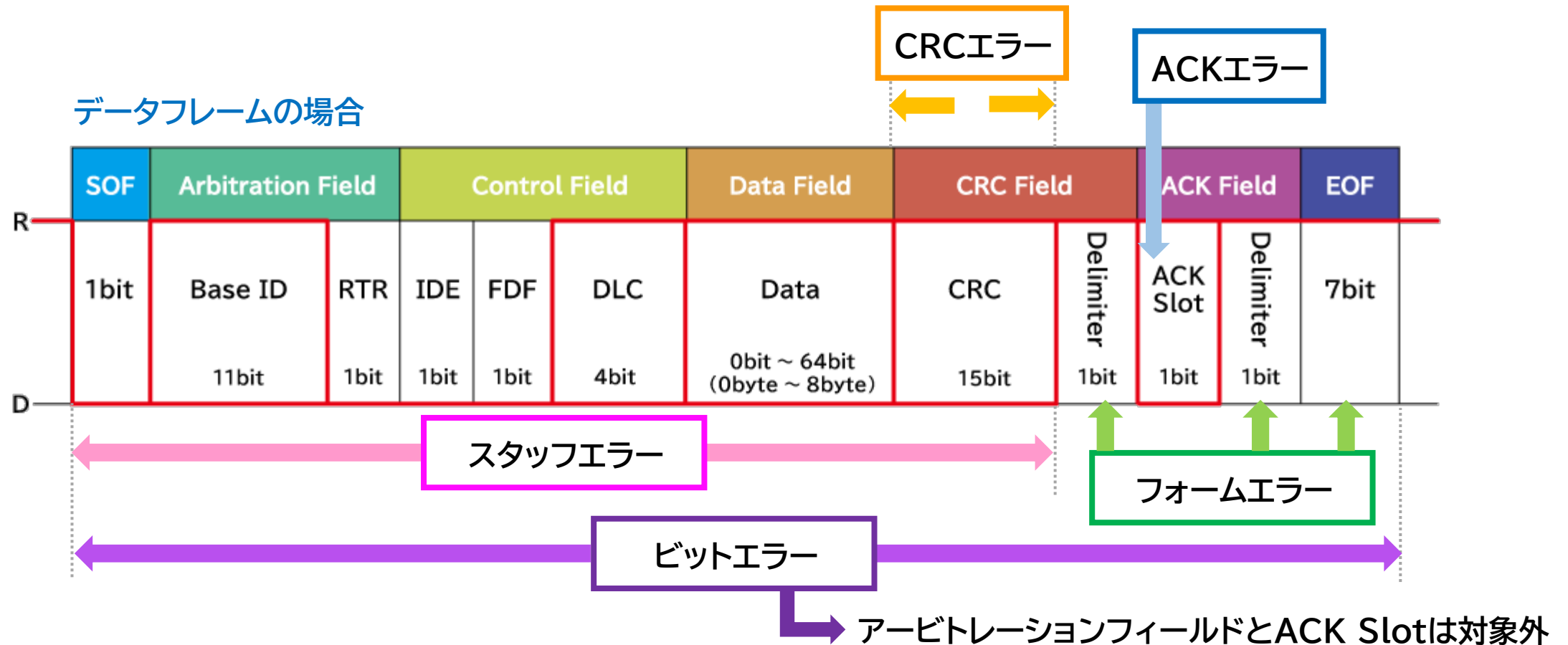
- ・「通信に参加できる正常な状態」
- ・ノードの初期状態
- ・エラーを検出すると積極的にエラー通知を行う
- ・エラーフレームでは、アクティブエラーフラグを送信する

- ・「エラーが頻発しているノード」
- ・エラーを検出しても積極的にエラー通知を行わない
- ・エラーフレームでは、パッシブエラーフラグを送信する
- ・インターフレームスペースにサスペンドトランスミッションが追加

- ・「故障したノード」
- ・バスから隔離され送信、受信ともに行えなくなる。

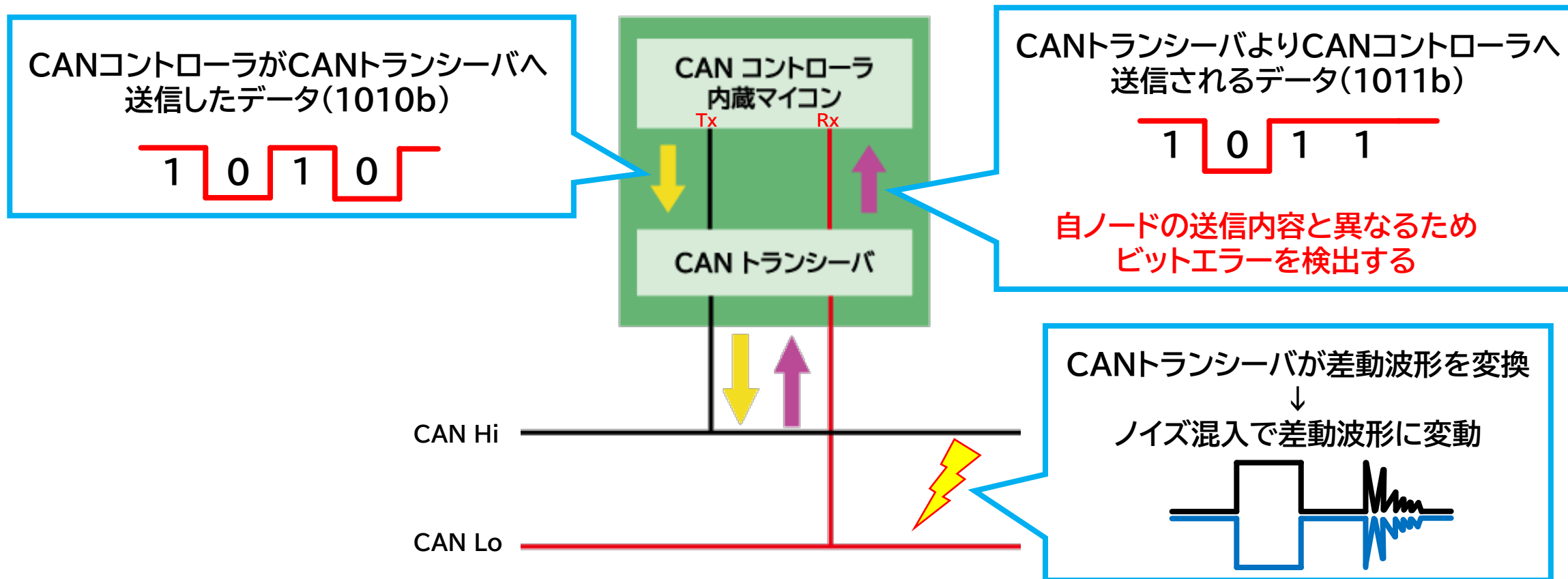
エラーを多発するノードは通信から隔離することで、他のノードの通信を妨げない(故障の封じ込め)仕組み

- CANには5種類のエラーがあり、複数のエラーが同時発生することがある。
- それぞれのエラーが検出される条件と検出箇所は規定で決まっている。



■検出ノード:送信

- 出力レベルとバスのレベルを比較して、**両者のレベルが不一致の場合に検出**
- ドミナント出力のスタッフビットは対象 送信時のアービトレーションフィールドとACKは対象外



■検出ノード:受信

- ・ビットスタッフが行われているはずのフィールドで**同一レベルが6bit連続した時に検出**

スタッフビットの付与/除去は
CANコントローラが自動で行う

送信したいデータ:000000b

0 0 0 0 0 1 0

CAN コントローラ
内蔵マイコン

Tx

Rx

CAN トランシーバ

除去すべきスタッフビットが
入っていない状態で受信される

0 0 0 0 0 0

本来必要なスタッフビットがないため
エラーを検出

本来バスに流れていると思っている波形

0 0 0 0 0 1 0

CAN Hi

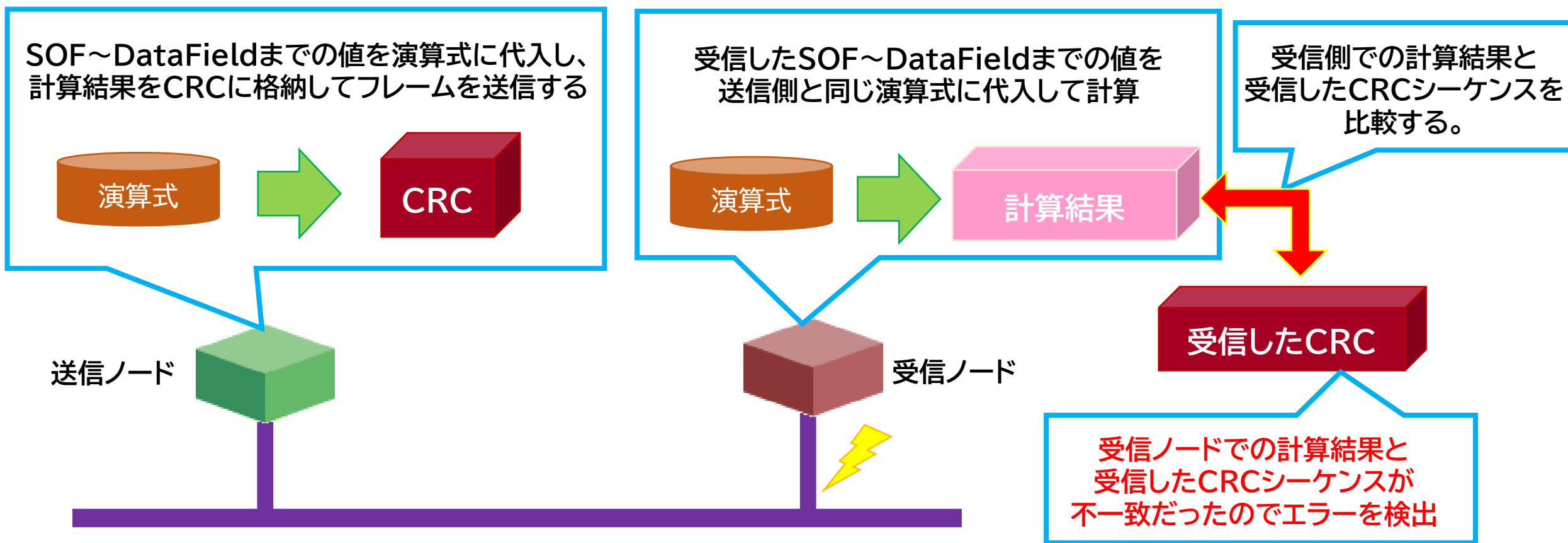
CAN Lo

実際にバスに流れた波形

0 0 0 0 0 0

■検出ノード:受信

- ・受信されたメッセージから算出したCRCの値と受信されたCRCシーケンスの値が異なる時に検出

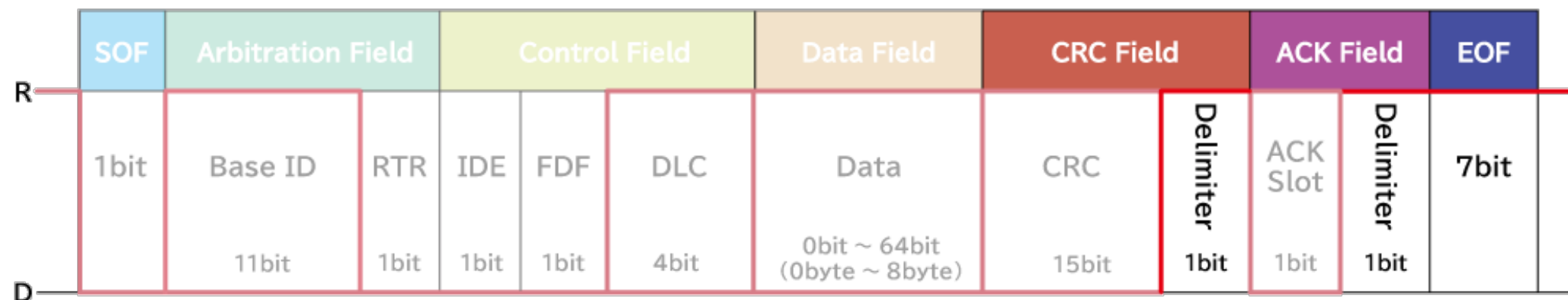


■検出ノード:送信/受信

- ・固定のフォーマットのビットフィールドに違反した時に検出

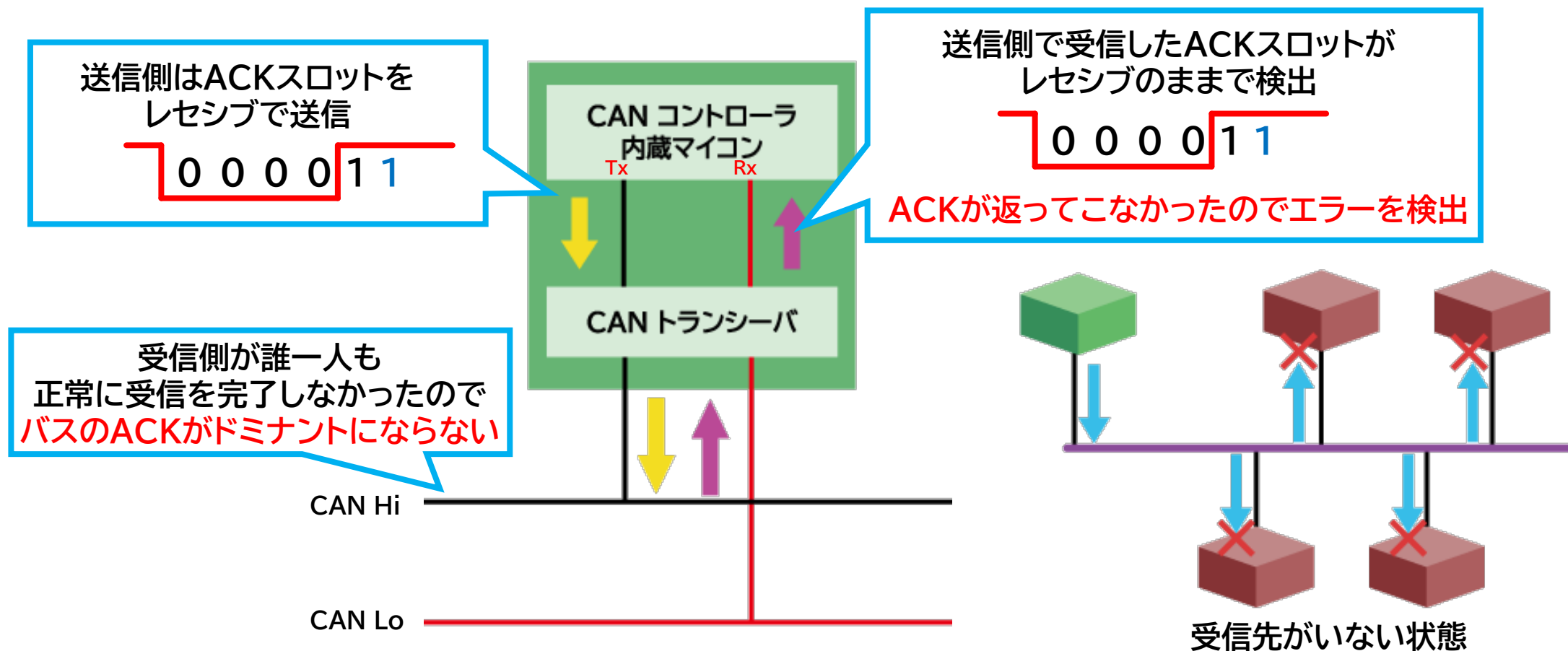
データフレームおよびリモートフレーム

CRC デリミタ / ACKデリミタ / EOFがドミナントレベルで出力されるとエラーを検出



■検出ノード:送信

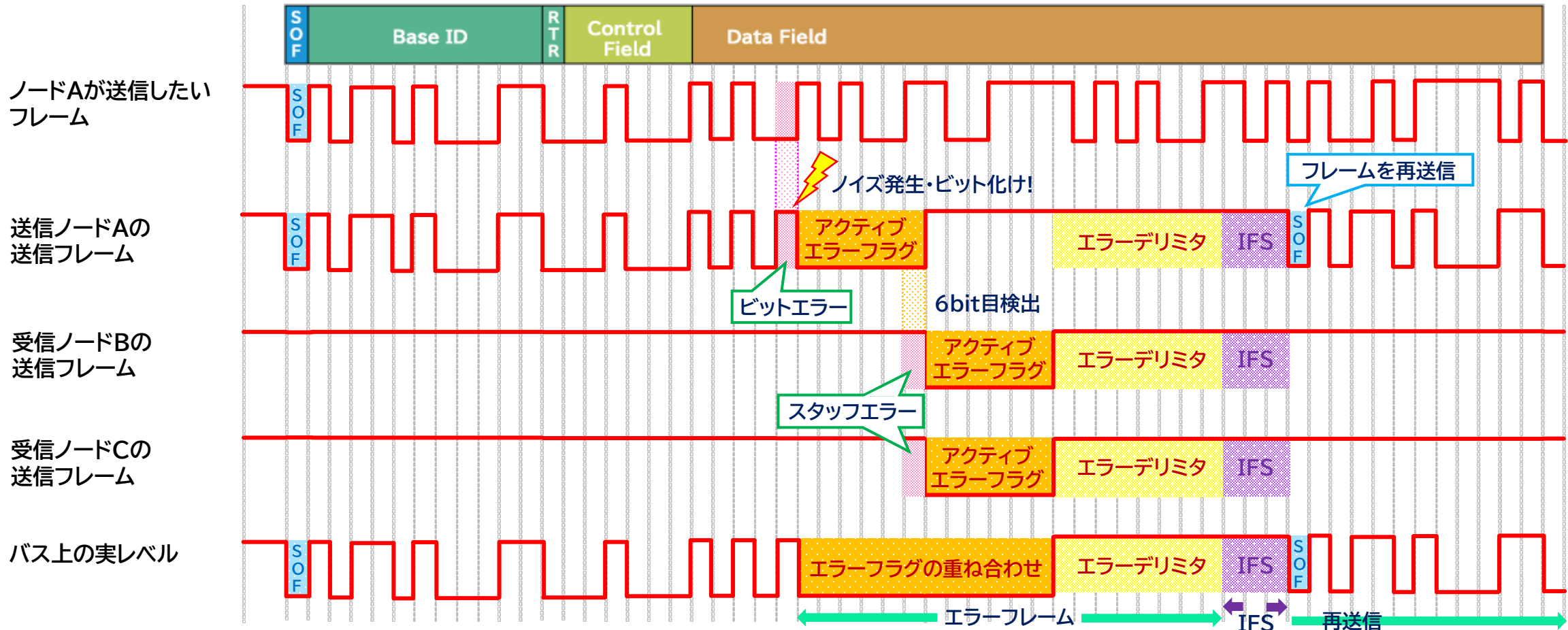
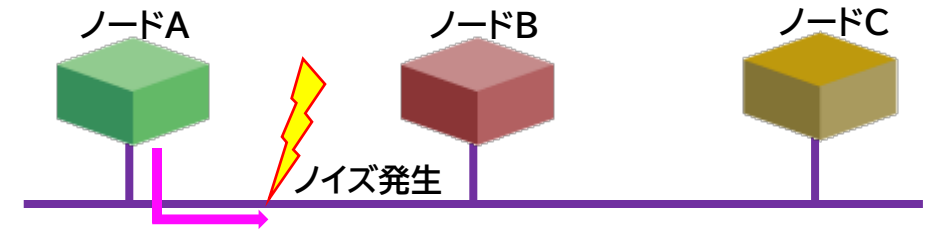
- ・送信ノードのACKスロットがレセシブレベルの場合に検出
= **ACKが返ってこなかった場合に検出**



■エラーフレームの送信シーケンス

-Error Frame Transmission sequence-

- ・エラーフレームは単独で存在せず、
他のフレームに寄生する形で送信される



■第3章まとめ

- 全てのノードはエラーステータス(エラー状態)を持っている
- エラーカウンタの値に応じてエラーステータスが遷移する
- 故障したノードは通信から隔離すること、故障の封じ込めを行うことで、バス全体の通信を保ち他のノードの通信を妨げない仕組みになっている
- CANには5種類のエラーがあり、複数のエラーが同時発生することがある
- エラーフレームは他のフレームに寄生する形で送信される